AI-BASED HYPER-LOCAL AVIATION WEATHER FORECASTING AND VERIFICATION SYSTEM FOR SRI LANKA

INDIVIDUAL THESIS

Ashan Dilakshana

(IT22269720)

Individual component: Wind Pattern Prediction, Runway-Oriented Wind Analysis and Pilot Portal

Group ID: R26-IT-132

Report submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed the use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Student name: Ranasingha R.A.A.D

Student ID: IT22269720

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor: Ms. Shashika Lokuliyana Signature: _____

Date: _____

Co-supervisor: Ms.Hansika Mahaadikara. Signature: _____

Date: _____

ABSTRACT

Aviation operations at airport terminals are strongly influenced by short-term wind variability, particularly during varying weather conditions. General meteorological forecasts provide wind speed and wind direction, but these values are not always expressed in a runway-oriented form that directly supports operational interpretation. This individual research addresses that gap through the development and evaluation of a machine learning-based hyper-local wind prediction component for Bandaranaike International Airport (VCBI), under varying local weather conditions. The project database contains historical VCBI observations covering 2019–2026. The implemented preprocessing and modelling pipeline prepares temporal wind features, represents wind direction using cyclic sine and cosine encoding, and supports XGBoost and Temporal Fusion Transformer forecasting. The final individual XGBoost evaluation reports an MAE of 1.0516 knots, RMSE of 1.4404 knots and R^2 of 0.7798 on 10,235 test observations. Wind direction is evaluated using circular angular error, with an angular MAE of 10.61 degrees. Predicted wind

vectors are transformed into runway-oriented headwind and crosswind components. For the evaluated Runway 04 configuration, headwind MAE and RMSE are 0.8833 and 1.2025 knots, while crosswind MAE and RMSE are 1.2027 and 1.7270 knots. The individual contribution also includes a pilot portal that supports flight planning, route information, weather briefing generation and live aviation weather map integration. The resulting component connects airport-level machine learning forecasting with deterministic runway-vector calculations and a pilot-oriented decision-support workflow. The system is intended as decision support and not as a replacement for official aviation meteorological services or pilot judgement.

Keywords: Aviation Weather Forecasting, Wind Prediction, local weather conditions, XGBoost, Crosswind, Headwind

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my research supervisor Ms. Shashika Lokuliyana for the guidance, technical feedback and continuous support provided throughout this individual research project. And the Co- supervisor Ms.Hansika Mahaadikara academic guidance helped refine the research problem, methodology, experimental evaluation and presentation of the final work.

I am deeply grateful to Mr. Ajith Wijemanna, Director General of the Department of Meteorology, Sri Lanka, for his support and for providing the professional meteorological context that was valuable to this research.

I would also like to sincerely thank Mr. Jeewan Karunaratna, Head Forecaster of the Department of Meteorology, Sri Lanka, for his valuable guidance, professional insights and support related to aviation weather forecasting and interpretation of meteorological information.

I am grateful to the Department of Information Technology at the Sri Lanka Institute of Information Technology for providing the academic environment, computing facilities and research framework required to complete this study.

I also acknowledge the wider research project team because the individual wind component operates within a larger hyper-local aviation weather forecasting and

verification platform. Shared data infrastructure, backend services and project-level system integration provided an important technical environment for the individual contribution. The wind prediction, runway-oriented analysis and pilot portal work documented in this thesis are nevertheless presented as the individual component.

Finally, I thank my colleagues, friends and family for their encouragement and support throughout the research, experimentation, software development and preparation of this thesis.

TABLE OF CONTENTS

Contents

DECLARATION AND SUPERVISOR CERTIFICATION ***1***

ABSTRACT ***2***

ACKNOWLEDGEMENTS ***3***

TABLE OF CONTENTS ***4***

LIST OF FIGURES ***9***

LIST OF TABLES ***9***

LIST OF ABBREVIATIONS ***10***

1 INTRODUCTION ***11***

1.1 Overall project context **11**

1.2 Individual component context **12**

1.3 Problem statement **12**

1.4 Research gap **13**

1.5 Aim and objectives **14**

1.6 Research questions or hypotheses **15**

1.7 Scope limitations and assumptions **16**

1.8 Individual contribution **17**

1.9 Thesis structure **17**

2 LITERATURE REVIEW AND RELATED WORK ***18***

2.1 Review method and scope **18**

2.2 Domain and theoretical foundations **18**

2.2.1 Aviation weather and airport-scale forecasting 18

2.2.2 seasonal weather influence 19

2.2.3 Wind as a vector 19

2.2.4 Circular direction representation 19

2.3 Related research **19**

2.4 Existing systems products and techniques **21**

2.5 Gap synthesis and conceptual framework **22**

3 RESEARCH METHODOLOGY ***22***

3.1 Research design **22**

3.2 Data participants or study materials **23**

3.3 Experimental or study procedure **24**

3.3.1 Data ingestion and preparation 24

3.3.2 Data cleaning 24

3.3.3 Feature engineering 24

3.3.4 XGBoost training 25

3.3.5 Temporal Fusion Transformer 26

3.3.6 Hybrid forecasting 26

3.3.7 Runway-oriented calculation 27

3.3.8 Alert logic 27

3.3.9 Forecast verification 27

3.4 Measures and evaluation metrics **28**

3.5 Data analysis **29**

3.6 Ethics privacy security and responsible AI **29**

3.7 Reproducibility and research artefacts **30**

4 PRODUCT AND SOLUTION DESIGN ***30***

4.1 Requirements **30**
4.1.1 Functional requirements 30
4.1.2 Non-functional requirements 32
4.2 Component architecture within the complete solution **32**
4.3 Design alternatives and decisions **33**
4.3.1 Linear versus cyclic wind direction 33
4.3.2 Tabular versus sequence modelling 33
4.3.3 Raw wind versus runway-relative output 33
4.3.4 Offline evaluation versus continuous verification 33
4.4 Detailed design **34**
4.5 Standards and professional practice **34**
5 IMPLEMENTATION INTEGRATION AND TESTING ***35***
5.1 Development environment and technology stack **35**
5.2 Component implementation **36**
5.2.1 Data pipeline implementation 36
5.2.2 Model implementation 36
5.2.3 Backend implementation 37
5.2.4 Pilot portal implementation 37
5.3 Integration with the other components **37**
5.4 Verification and software testing **38**
5.4.1 Data tests 38
5.4.2 Model tests 38
5.4.3 Physics and transformation tests 38
5.4.4 Portal tests 38
5.5 Deployment and operation **38**
6 RESULTS AND VALIDATION ***39***
6.1 Research results **39**
6.1.1 Model performance interpretation 40
6.1.2 Wind-direction results **40**
6.1.3 Runway-oriented wind results **41**

6.1.4 Alert-threshold results **42**

6.1.5 Project-level TFT and hybrid evidence **42**

6.2 Product and technical results **42**

6.3 User or stakeholder validation **43**

6.4 Integration results **43**

6.5 Summary of findings **44**

7 DISCUSSION ***44***

7.1 Interpretation of findings **44**

7.2 Comparison with literature baselines and existing solutions **45**

7.3 Research contribution **46**

7.4 Product contribution **46**

7.5 Limitations and threats to validity **46**

7.6 Implications **47**

8 COMMERCIALISATION AND SUSTAINABILITY ***48***

8.1 Value proposition and target users **48**

8.2 Market and competitive position **48**

8.3 Business or sustainability model **48**

8.4 Scalability intellectual property and compliance **49**

8.5 Contribution to the Sustainable Development Goals **49**

9 CONCLUSION AND FUTURE DIRECTIONS ***49***

9.1 Conclusion **49**

9.2 Achievement of objectives **50**

9.3 Individual research and product contributions **51**

9.4 Future directions **51**

REFERENCES ***52***

GLOSSARY ***54***

APPENDICES *56*

Appendix A: Research instruments or extended results **56**

A.1 Extended evaluation summary 56
A.2 Verification-data handling note 57
A.3 Suggested final figures 57
A.4 Pilot portal evidence 57
A.5 Individual-component boundary and responsibility 59
A.6 Evidence register and claim status 60
A.7 Data source and observation meaning 61
A.8 Data dictionary: temporal fields 62
A.9 Data dictionary: wind variables and units 63
A.10 Data dictionary: atmospheric variables 64
A.11 Cleaning decisions and their consequences 65
A.12 Lag features and rolling summaries 66
A.13 Cyclic encoding of wind direction 67
A.14 Hour-of-day encoding 68
A.15 XGBoost model: methodological explanation 69
A.16 Hyperparameter search and reproducibility 70
A.17 Train-test splitting and temporal leakage 71
A.18 Temporal Fusion Transformer configuration 72
A.19 Sequence dataset construction 73
A.20 Hybrid forecast concept and interpretation 74
A.21 Mean absolute error: definition and use 75
A.22 Root mean squared error: definition and use 76
A.23 Coefficient of determination R-squared 77
A.24 Circular angular error for direction 78
A.25 Runway-relative wind equations 79
A.26 Runway selection logic and boundary cases 80
A.27 Crosswind alert logic 81
A.28 Forecast target time and horizon semantics 82
A.29 Verification record schema 83
A.30 Verification export: interpretation limits 84
A.31 Backend route separation 85
A.32 Pilot portal workflow and flight-plan data 86
A.33 Scheduling and data refresh 87
A.34 Model retraining and model governance 88
A.35 API error handling and input validation 89

A.36 Pilot-facing communication and safe interpretation 89
A.37 Software test strategy 89
A.38 Test cases: runway calculations 89
A.39 Test cases: data and model pipeline 89
A.40 Test cases: API and pilot workflow 89
A.41 Non-functional requirements and evidence 89
A.42 Traceability from objectives to evidence 89
A.43 Limitations of the available evidence 89
A.44 Threats to validity 89
A.45 Reproducibility checklist 89
A.46 Data integrity and database consistency 89
A.47 Missing data and forecast availability 89
A.48 Model comparison protocol 89
A.49 Operational monitoring indicators 89
A.50 Risk register for the wind component 89
A.51 Privacy, access control and responsible use 89
A.52 Sustainability and maintainability of the implementation 89
A.53 Commercial and deployment boundary 89
A.54 Future evaluation plan 89
A.55 Reproducible pseudocode for the wind workflow 89
A.56 Research reproducibility and version control 89
A.57 Interpretation of the recorded XGBoost results 89
A.58 Interpreting no observed extreme-threshold cases 89
A.59 Research questions and evidence mapping 89
A.60 Glossary of component-specific terms 90

Appendix B: Academic integrity and AI use disclosure 90

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BIA	Bandaranaike International Airport
CW	Crosswind
DSR	Design Science Research
HW	Headwind
ICAO	International Civil Aviation Organization
MAE	Mean Absolute Error
METAR	Meteorological Aerodrome Report
ML	Machine Learning
MLOps	Machine Learning Operations
NWP	Numerical Weather Prediction
QNH	Altimeter Setting / Pressure at Sea-Level Reference
RMSE	Root Mean Square Error
R^2	Coefficient of Determination
TAF	Terminal Aerodrome Forecast
TFT	Temporal Fusion Transformer
VCBI	ICAO identifier for Bandaranaike International Airport

Table 1: List of Abbreviations

1 INTRODUCTION

1.1 Overall project context

Weather is a critical operational factor in aviation because aircraft performance, runway selection, route planning and flight safety are influenced by atmospheric conditions. Among the most operationally relevant variables are wind speed and wind direction. The same wind speed can have very different consequences depending on whether the wind is aligned with a runway, opposed to the runway direction, or approximately perpendicular to the runway. Consequently, an aviation weather decision-support system must do more than report a numerical wind value; it should help users interpret the wind in the context of aircraft operations.

The wider research project is an AI-based hyper-local aviation weather forecasting and verification system for Sri Lankan weather patterns. Its central idea is to move from broad weather information toward airport-level forecasting using historical observations and machine learning. The project focuses on Bandaranaike International Airport (BIA), represented by the ICAO identifier VCBI. The system combines meteorological observations, predictive models, verification mechanisms and web-based decision-support interfaces.

The research proposal identifies a particular challenge in tropical and seasonal weather-affected aviation environments: weather patterns can change over short time periods, while generalized forecasts may not capture localized airport conditions with sufficient operational detail. The proposal therefore motivates a hyper-local approach that uses airport observations and converts predicted meteorological information into aviation-relevant indicators. The individual work documented in this thesis focuses specifically on the wind component of that wider system.

The individual component is aligned with the Topic Assessment Form allocation for the researcher. The assigned work includes development of a local weather conditions wind pattern prediction model, analysis of historical wind speed and direction, evaluation of forecast performance, runway-oriented conversion into headwind and crosswind components, and development of the pilot portal. These

responsibilities define the boundary between the individual contribution and the other weather-prediction components developed by the wider project.

1.2 Individual component context

The individual component addresses wind behaviour around VCBI during the local weather conditions. Wind observations are treated as a time-dependent sequence rather than as independent records. The implemented feature-engineering pipeline therefore retains temporal information, creates recent-history features, and represents circular variables using trigonometric transformations.

The forecasting layer supports both a conventional gradient-boosting model and a sequence-learning model. XGBoost is used for structured tabular prediction, while a Temporal Fusion Transformer is used to model temporal relationships. The project also contains a hybrid three-hour forecasting route that combines model outputs. The individual thesis reports the reproducible XGBoost evaluation separately from approximate project-level TFT and hybrid results when exact paired evaluation values are not available.

A second major part of the individual component is operational interpretation. The backend contains a deterministic calculation that takes wind speed, wind direction and runway heading and decomposes the wind vector into headwind and crosswind. The system considers VCBI Runway 04 with heading 040 degrees and Runway 22 with heading 220 degrees. It can compare both runway directions and recommend the runway with the larger headwind component within the implemented logic.

The third part is the pilot-facing interface. The backend pilot router accepts departure, destination, departure time, area, flight levels, selected map products, pilot reference and flight number. It can create a flight plan, obtain associated flight information, generate a briefing document and retrieve aviation weather maps. This allows the predictive output to be consumed as part of a broader flight-planning workflow.

1.3 Problem statement

Although wind speed and wind direction are standard aviation weather variables, a raw wind forecast does not directly tell a pilot how much of the wind will act along

or across the runway. This creates an interpretation step between meteorological forecasting and aviation decision support. The research proposal identifies this problem explicitly: existing systems can provide general meteorological variables but rarely convert those predictions into runway-oriented indicators such as headwind and crosswind.

The problem is more important under seasonal weather conditions because wind patterns can vary over time. A localized forecast that is accurate on average but difficult to interpret operationally may still provide limited decision-support value. Therefore, the individual research problem is defined as the lack of an integrated airport-level wind prediction and runway-oriented interpretation pathway for local weather conditions conditions at VCBI.

The problem is addressed through a chain of processes: historical observation preparation, temporal feature engineering, machine learning prediction, circular wind-direction handling, runway vector decomposition, forecast verification and pilot-oriented presentation. The objective is not to replace official aviation meteorology, but to provide a technically evaluated decision-support component.

1.4 Research gap

The literature reviewed in the research proposal demonstrates that machine learning has been applied to rainfall forecasting, short-term weather prediction, precipitation nowcasting, ensemble post-processing and automated aviation forecasting. However, many studies focus on general atmospheric variables, broad geographic domains or forecast generation rather than airport-specific operational interpretation.

The proposal identifies a more specific gap: machine-learning wind prediction research is often separated from aviation decision-support requirements. A wind model may produce a speed and direction, but an aviation user may still need to calculate runway-relative components manually. The proposed individual work addresses this gap by linking the forecast to runway geometry and presenting headwind and crosswind information through a pilot-facing portal.

A second methodological gap concerns wind direction. Direction is a circular variable, meaning that 0 degrees and 360 degrees represent the same physical direction. Treating direction as a normal linear number can produce large artificial errors around the north boundary. The implemented approach uses sine and cosine encoding and evaluates the reconstructed direction using circular angular error.

A third gap concerns continuous verification. A forecast is meaningful only when its target-time prediction can be compared with an actual observation. The project stores forecast target timestamps and verified forecast records, creating a foundation for repeated forecast-to-observation assessment.

1.5 Aim and objectives

The main aim of this individual research is to develop and evaluate a machine learning-based hyper-local wind prediction component for local weather conditions conditions at Bandaranaike International Airport and convert the predicted wind information into runway-oriented aviation indicators that can be consumed through a pilot decision-support portal.

The objectives were derived from the individual allocation in the Topic Assessment Form and the research proposal.

No.	Individual objective
O1	Collect and analyze historical VCBI wind speed and wind direction observations relevant to local weather conditions conditions.

O2	Identify temporal patterns and engineer features that improve short-term wind prediction.
O3	Develop and evaluate machine learning models for short-term wind forecasting.
O4	Represent wind direction using circular features and evaluate direction using an appropriate angular metric.
O5	Convert predicted wind vectors into runway-oriented headwind and crosswind components.
O6	Implement warning logic for high crosswind conditions and expose the results through the backend.
O7	Support prediction-to-observation verification using target timestamps and subsequent meteorological observations.
O8	Develop a pilot portal workflow that provides flight planning, route information and aviation weather briefing support.

Table 1: Individual Research Objectives

1.6 Research questions or hypotheses

The research is evaluated through the following questions:

1. RQ1: How accurately can short-term wind speed at VCBI be predicted using the implemented machine learning approach?
2. RQ2: Does cyclic representation provide a meaningful method for predicting and reconstructing wind direction?

3. RQ3: How accurately can the predicted wind be converted into runway-relative headwind and crosswind values?
4. RQ4: Can forecast records be associated with subsequent observations in a way that supports repeatable verification?
5. RQ5: Can the resulting information be integrated into a pilot-facing workflow without requiring the user to manually calculate runway wind components?

The principal quantitative expectation is that the model should achieve an average wind-speed error low enough to be operationally useful as a decision-support signal. The project paper discusses a target below approximately 2 knots and an operational error criterion around 2.5 knots. These values are treated as engineering reference points rather than formal certification limits.

1.7 Scope limitations and assumptions

The research scope is restricted to the wind-focused component of the wider hyper-local aviation weather system. The primary geographical scope is VCBI. The historical database used by the project covers 2019–2026. The principal forecast horizon examined by the individual component is three hours, while the implemented backend also provides a one-hour route.

The system assumes that the meteorological observations used for training are sufficiently representative of the local airport environment and that future METAR observations can serve as an observation reference for forecast verification. It also assumes that the runway headings configured in the backend represent the intended runway orientations for the analysis.

The system is not a certified aviation weather service and should not be interpreted as a replacement for official METAR/TAF products, meteorological forecaster judgement, aircraft-specific performance limits or air-traffic operational procedures. The pilot portal is therefore a decision-support interface.

A specific validation limitation is the rarity of high crosswind cases in the available test sample. The test set contained no observations above the implemented 15-knot warning threshold or 25-knot danger threshold. Consequently, the research can verify that no false alert occurred in the evaluated sample, but it cannot establish sensitivity or recall for rare hazardous crosswind events.

1.8 Individual contribution

The individual contribution covers the wind prediction pathway and the pilot interaction functions assigned to the researcher. This includes data preparation for wind forecasting, temporal feature engineering, circular wind-direction encoding, XGBoost and TFT wind forecasting workflows, evaluation of wind-speed and direction performance, runway-oriented vector calculations, crosswind alert logic, forecast verification support and pilot portal functions.

Shared infrastructure is not presented as an individual invention. The wider project provides a common database, FastAPI architecture, live METAR acquisition and project-level deployment environment. The thesis instead explains how the individual wind component uses these facilities and what additional research and implementation work was carried out for the wind-focused objective.

The distinction is important for an individual thesis because the complete project contains multiple weather components. The thesis therefore avoids attributing temperature, cloud/visibility, QNH or dewpoint research to the individual contribution except where these variables are used as input features or integration dependencies for the wind module.

1.9 Thesis structure

Chapter 1 introduces the research context, problem, gap, objectives, questions, scope and individual contribution. Chapter 2 reviews the theoretical foundations and related research. Chapter 3 describes the research methodology, data, preprocessing, feature engineering, model training, evaluation and responsible-AI considerations. Chapter 4 presents the product and solution design. Chapter 5 explains implementation, integration and testing. Chapter 6 reports the quantitative and technical results. Chapter 7 interprets the findings, compares them with related work and discusses limitations. Chapter 8 evaluates commercialisation and sustainability. Chapter 9 concludes the thesis and identifies future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review was derived from the sources already used in the research proposal and the wider project IEEE paper. The review scope was intentionally narrowed to material relevant to the individual wind component: aviation weather hazards, Sri Lankan and tropical weather forecasting, machine learning for short-term atmospheric prediction, time-series deep learning, wind-direction representation, aviation decision support and forecast verification.

The review does not attempt to treat every weather forecasting method as equally relevant. Instead, studies were grouped according to the problem they help explain. Sri Lankan seasonal weather studies establish the local relevance of data-driven forecasting. Aviation meteorology studies establish the operational importance of weather hazards. Machine-learning studies establish the suitability of nonlinear and temporal models. Automated aviation forecasting studies establish the movement toward AI-assisted decision support.

The final gap is synthesized from the difference between these strands. Existing work demonstrates that machine learning can predict atmospheric variables and that aviation users need localized weather information, but the individual research focuses specifically on connecting predicted wind with runway-relative operational quantities and pilot-facing delivery.

2.2 Domain and theoretical foundations

2.2.1 Aviation weather and airport-scale forecasting

Aviation weather forecasting differs from general weather forecasting because the relevant spatial scale can be very small. A forecast that is suitable for a region may not represent the conditions at an individual airport runway. Local wind observations can therefore be valuable for short-horizon prediction and verification.

The project uses METAR observations as an important observation source. METAR provides standardized airport weather information, including wind, visibility, cloud and other atmospheric parameters. The individual wind module uses the local

observation history to construct features and later compares forecasts with future observations.

2.2.2 seasonal weather influence

Sri Lankan weather is influenced by seasonal seasonal weather circulation and inter-varying weather conditions. The research proposal specifically positions the project around weather-aware aviation forecasting because the atmospheric environment can change across seasons. The individual component focuses on local weather conditions wind behaviour rather than attempting to build a universal all-season model without seasonal context.

2.2.3 Wind as a vector

Wind is represented by magnitude and direction. For runway operations, the useful decomposition is into a component parallel to the runway and a component perpendicular to it. If V is wind speed and theta is the angular difference between wind direction and runway heading, the headwind component can be expressed as V cos(theta), while the signed crosswind component can be expressed as V sin(theta). The implementation uses the absolute value of the crosswind component when reporting its magnitude.

2.2.4 Circular direction representation

Wind direction has circular topology. Direct numerical regression can create a discontinuity at 360 degrees. To avoid this, the implementation creates sine and cosine features using the direction angle. The two-dimensional representation preserves continuity because directions near 0 degrees and 360 degrees map to nearby points on the unit circle.

2.3 Related research

Study / source	Main relevance	Relationship to individual work
[1] Wickramagamage	Neural-network-based seasonal weather rainfall forecasting in Sri Lanka.	Supports the use of data-driven methods in Sri

		Lankan weather forecasting.
[2] Jayawardena	Machine-learning-based short-term weather prediction in Sri Lanka.	Provides local evidence for ML-based short-horizon prediction.
[3] Wang	Aviation weather hazards and their mechanisms.	Supports the importance of localized aviation weather analysis.
[4] Jayasinghe	Lightning climatology at Bandaranaike International Airport.	Demonstrates value of airport-specific climatological analysis.
[5] Sprenger and co-author	Review of high-impact weather for aviation.	Provides aviation meteorology and safety context.
[6] Jeong	AI framework for automated TAF generation.	Demonstrates AI-assisted aviation weather support.
[7] Shi et al.	ConvLSTM for precipitation nowcasting.	Demonstrates deep temporal modelling of atmospheric variables.
[8] Chen	Review of deep learning for weather forecasting.	Supports deep-learning approaches for atmospheric prediction.
[9] Bauer et al.	ML integration challenges in weather/climate modelling.	Highlights data, scale and model-integration challenges.

[10] Rasp and Lerch	Neural-network post-processing of ensemble forecasts.	Demonstrates ML use for forecast-error correction.

Table 2.1: Summary of Selected Related Studies

Table 1 Related research

The selected literature indicates that the use of machine learning for weather forecasting is not itself a new concept. The individual contribution is instead situated at the intersection of airport-scale forecasting and aviation-specific interpretation. This distinction is important when describing novelty: the thesis does not claim to invent XGBoost, TFT or wind-vector mathematics. The contribution is the integration, evaluation and operationalization of these techniques for the defined VCBI wind problem.

2.4 Existing systems products and techniques

Conventional aviation weather workflows use standardized observations such as METAR and forecasts such as TAF. These products remain authoritative operational information sources. Machine learning can be used as a supplementary forecasting or decision-support mechanism, but its outputs need verification against actual observations.

The wider project architecture adds automated acquisition and storage, predictive models, verification tables and a web dashboard. The wind module specifically exposes one-hour, three-hour, TFT and hybrid routes. The backend also calculates runway-relative wind and generates status messages when crosswind exceeds configured thresholds.

The main limitation of a meteorological-only presentation is that the user still has to mentally map direction and speed to the runway. The proposed solution makes this transformation explicit. This improves interpretability without changing the underlying physics of the wind vector.

2.5 Gap synthesis and conceptual framework

The reviewed literature supports three conclusions. First, local weather observations are valuable for short-term airport forecasting. Second, machine learning can model nonlinear and temporal relationships in atmospheric data. Third, aviation users require information that is directly interpretable in the context of operational conditions.

The conceptual framework therefore consists of five linked layers: observation, prediction, physical interpretation, verification and decision support. The observation layer supplies historical and live METAR data. The prediction layer uses engineered features and ML models. The physical interpretation layer decomposes wind relative to runway orientation. The verification layer compares forecast target values with subsequent observations. The decision-support layer presents the result through the pilot portal.

Figure 4.1 Individual wind component architecture.

Figure 4.1 shows the conceptual relationship between the individual component and the wider system. The architecture is intentionally modular so that the wind pathway can be evaluated independently while still consuming common project services.

3 RESEARCH METHODOLOGY

3.1 Research design

The research follows a Design Science Research (DSR) approach combined with experimental evaluation. DSR is appropriate because the work creates a technical artifact intended to address a practical problem. The artifact includes a predictive model, data-processing pipeline, runway-analysis functions and a pilot-facing interaction layer.

The experimental component evaluates the forecasting artifact using historical observations. The implementation component establishes how the model can operate as part of a continuously running system. This combination allows the thesis to evaluate both research performance and practical integration.

The research process can be summarized as: identify the aviation wind problem; define the individual scope; prepare historical observations; engineer features; train candidate models; evaluate predictions; transform predictions into runway components; verify forecasts against future observations; integrate the output into the pilot portal; and analyze limitations and future improvements.

3.2 Data participants or study materials

The research does not use human participants as the primary experimental data source. The principal study material is airport meteorological observation data. The project database uses a weather_data table containing timestamp information and meteorological variables including visibility, weather, clouds, wind direction, wind speed, dry temperature, dew point, relative humidity and QNH.

The final project database contains 121,407 historical weather records covering 2019–2026. The three-hour wind pipeline applies preprocessing and temporal feature construction. The resulting model-ready dataset contains 51,175 valid rows according to the recorded processing results used for this thesis. The XGBoost evaluation uses an 80/20 split, corresponding to 40,940 training observations and 10,235 test observations.

Dataset item	Value	Role
Raw weather records	121,407	Historical airport observations available in the project database
Historical coverage	2019–2026	Long-term local temporal context
Cleaned 3-hour modelling rows	51,175	Rows remaining after preprocessing and temporal feature construction
XGBoost training rows	40,940	Model fitting subset

XGBoost test rows	10,235	Unseen evaluation subset
TFT valid sequence samples	67,563	Prepared sequence-learning sample count reported for the TFT pipeline

Table 3.1: Dataset and Modelling Data Summary

Table 2 Dataset and modelling data summary

3.3 Experimental or study procedure

3.3.1 Data ingestion and preparation

The three-hour training script connects to the SQLite database and queries the weather_data table in database order. Database field names are mapped to the column names expected by the unified weather pipeline. This mapping allows the same feature-engineering logic to be used for historical training and later inference.

3.3.2 Data cleaning

The data-processing pipeline is designed to handle missing and invalid values before feature creation. Timestamp fields are reconstructed from year, month, date and UTC time. Records are ordered chronologically. Rows that cannot form valid timestamps are removed. Temporal feature generation then creates lag and rolling variables, which naturally creates unavailable values at the beginning of the series; these are excluded from the final model-ready data.

3.3.3 Feature engineering

The feature-engineering implementation creates wind-direction sine and cosine values, dew-point depression, hour-of-day sine and cosine values, a three-hour lag of wind speed, a six-hour rolling mean of wind speed, a three-hour QNH lag and QNH change over three hours. These features are designed to provide the model with both instantaneous atmospheric state and recent temporal context.

Feature	Construction	Purpose
Wind Dir_Sin	sin(direction in radians)	Circular representation of direction
Wind Dir_Cos	cos(direction in radians)	Circular representation of direction
Dew_Point_Depression	Dry temperature − dew point	Represents moisture/air-mass state
Hour_Sin / Hour_Cos	Cyclic hour-of-day encoding	Represents daily periodicity
Wind speed lag 3h	Wind speed shifted by three hours	Recent historical wind state
Wind rolling mean 6h	Six-hour time-based rolling mean	Short-term wind trend
QNH lag 3h	Pressure shifted by three hours	Recent pressure state
QNH change 3h	Current QNH − three-hour lag	Pressure tendency

Table 3.2: Engineered Wind Features

3.3.4 XGBoost training

The XGBoost training implementation uses an 80/20 train-test split with random_state 42. GridSearchCV is used to tune the number of estimators, tree depth, learning rate, subsampling and column subsampling. The search uses three-fold cross-validation and negative MAE as the scoring objective. The best estimator is then evaluated on both the training and unseen test partitions.

Hyperparameter	Values searched
n_estimators	300, 500, 1000
max_depth	5, 7, 9
learning_rate	0.01, 0.05
subsample	0.8, 1.0
colsample_bytree	0.8, 1.0
Cross-validation	3-fold
Selection metric	Negative MAE

Table 3.3: XGBoost Hyperparameter Search Space

3.3.5 Temporal Fusion Transformer

The TFT implementation uses PyTorch Forecasting and Lightning. The configured sequence dataset uses a maximum encoder length of 48 time steps and a maximum prediction length of three steps. The model configuration uses learning rate 0.03, hidden size 32, two attention heads, dropout 0.15, hidden continuous size 16 and gradient clipping of 0.1. Early stopping monitors validation loss with a patience of three epochs.

The TFT model is useful as a complementary approach because it explicitly models temporal context rather than relying only on manually constructed tabular lag features. The implementation uses MAE as the training loss. The project paper reports approximately 1-knot wind error for the TFT route; the exact independent test result should be recalculated from a clean paired test set if it is required for final submission.

3.3.6 Hybrid forecasting

The wider project contains a hybrid three-hour route that combines XGBoost and TFT predictions. The rationale is that a tree-based model and a sequence model may capture different aspects of the same signal. The project paper reports hybrid wind performance below approximately one knot MAE. The thesis treats this as a project-

level reported result unless a complete paired hybrid evaluation file is available for the individual test partition.

3.3.7 Runway-oriented calculation

The backend function calculate_aviation_winds takes wind speed, wind direction and runway heading. It computes the directional difference in radians and applies sine and cosine decomposition. The absolute crosswind magnitude is reported, while the headwind component retains its sign so that headwind and tailwind can be distinguished.

Figure 4.2 Runway-oriented wind vector decomposition.

The implementation compares Runway 04 at 040 degrees and Runway 22 at 220 degrees. The suggest_best_runway function compares the calculated headwind values and returns the runway with the larger headwind under the current implementation.

3.3.8 Alert logic

The backend includes threshold-based alert logic. A crosswind above 15 knots produces a high-crosswind warning, while a crosswind above 25 knots produces a severe-crosswind status. These values are software thresholds used by the project and should not be described as universal regulatory limits without a separately verified aviation standard.

3.3.9 Forecast verification

The database defines forecast and verified-forecast structures. A forecast stores a target time and predicted weather quantities. The VerifiedForecast structure stores the target time together with observed values and calculated headwind and crosswind. This architecture enables a forecast made at time t for a future target time to be compared with the observation that later becomes available at that target time.

The verification text supplied with the project contains examples of one-hour, three-hour, TFT and hybrid predictions compared against later actual wind observations. It also contains some pending-live-data records and repeated timestamps. These

characteristics mean that raw verification exports should be deduplicated and checked before being used as official thesis metrics.

3.4 Measures and evaluation metrics

Metric	Definition	Why used
MAE	Mean absolute difference between predicted and actual values.	Easy to interpret in knots and directly expresses average error magnitude.
RMSE	Square root of mean squared error.	Penalizes larger forecast errors more strongly.
R^2	Coefficient of determination.	Indicates how much variation in the target is explained by the model.
Angular MAE	Mean circular angular deviation.	Appropriate for wind direction because direction wraps at 360°.
Headwind MAE/RMSE	Error of runway-parallel wind component.	Measures aviation-oriented transformation performance.
Crosswind MAE/RMSE	Error of runway-perpendicular wind component.	Measures the accuracy of an operationally important wind quantity.

Table 3.4: Evaluation Metrics Used in the Study

The evaluation should always state the data period, sample count and split strategy together with each metric. This is particularly important for time-series work because two MAE values cannot be assumed directly comparable if they were obtained from different periods, preprocessing versions or test-set definitions.

3.5 Data analysis

Data analysis is performed at three levels. First, descriptive analysis examines the structure of the historical dataset and the availability of wind variables. Second, model analysis evaluates the numerical forecast errors. Third, aviation interpretation analysis evaluates the transformed headwind and crosswind values and the operation of threshold-based alerts.

The final XGBoost test results used in this thesis are MAE 1.0516 knots, RMSE 1.4404 knots and R^2 0.7798. These values indicate that the model explains a substantial portion of the variation in the evaluated test sample while keeping the average absolute error close to one knot.

For wind direction, the reported angular MAE is 10.61 degrees over the 10,235-observation evaluation set. The use of angular error is essential because ordinary absolute degree error would overstate errors around the north boundary.

3.6 Ethics privacy security and responsible AI

The core research data are environmental observations and do not require collection of sensitive personal information for model training. Nevertheless, the pilot portal introduces user accounts and flight-plan information. The database model contains a User table with username, hashed password, role, name, email, organisation, station and account status. This requires responsible handling of authentication data and appropriate access controls.

The system also has safety implications because users may interpret forecasts as operational advice. The interface must therefore communicate that model output is decision support. Predictions should not be represented as guaranteed conditions. Verification results and data freshness should be visible where possible, and the system should retain the distinction between forecast information and authoritative aviation products.

The thesis does not claim a formal human-subject usability study because no validated participant study evidence is included in the supplied project artefacts. Portal usability is therefore discussed as technical and workflow validation rather than as statistically significant user research.

3.7 Reproducibility and research artefacts

The main reproducibility chain is: weather_data.db → database field mapping → UnifiedWeatherPipeline → processed_seasonal weather_data_3h.csv → XGBoost/TFT training → saved model → API inference → forecast record → future METAR verification → runway-vector calculation → pilot portal presentation.

The repository contains separate folders for preprocessing, wind models, backend API routes, MLOps retraining and pilot functions. This separation supports maintainability and allows the individual contribution to be traced to specific implementation areas. For final submission, the thesis should include selected code excerpts only where they help explain a technical decision; the complete repository should remain the supporting software artefact.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Functional requirements

ID	Requirement	Rationale
FR01	Acquire or receive airport weather observations.	Provides the input required for hyper-local forecasting.
FR02	Prepare historical wind data and temporal features.	Makes the raw observations suitable for modelling.
FR03	Generate one-hour and three-hour wind forecasts.	Supports short-horizon operational planning.
FR04	Represent wind direction cyclically.	Avoids the 0°/360° discontinuity.

FR05	Calculate runway headwind and crosswind.	Converts meteorological output into aviation information.
FR06	Select the more favourable runway using headwind comparison.	Provides a deterministic runway suggestion within the software logic.
FR07	Generate crosswind warning/danger statuses.	Highlights potentially important wind conditions.
FR08	Store forecasts with target timestamps.	Enables future verification.
FR09	Compare forecast values with later observations.	Provides model monitoring and evaluation.
FR10	Allow pilots to submit flight-plan information.	Connects forecast information with a flight context.
FR11	Generate weather briefing documents and maps.	Provides structured information for flight planning.
FR12	Maintain a record of pilot flight plans/documents.	Supports retrieval of previously generated information.

Table 4.1: Functional Requirements

4.1.2 Non-functional requirements

Category	Requirement
Accuracy	Forecast errors must be measurable and reported with appropriate metrics.
Availability	The wind prediction service should remain available as an independent module.
Maintainability	Models, preprocessing and API routes should be modular.
Traceability	Forecasts and verification records should retain timestamps.
Security	Pilot authentication information should be protected and passwords stored as hashes.
Usability	Outputs should be understandable without requiring manual vector calculations.
Scalability	The design should allow additional airports and new models to be added.
Safety	Forecasts must be clearly presented as decision support rather than certified operational clearance.

Table 4.2: Non-Functional Requirements

4.2 Component architecture within the complete solution

The architecture separates data acquisition, prediction, physical transformation, verification and user interaction. This separation is useful because each layer has a different responsibility. Data acquisition is concerned with freshness and

completeness; machine learning is concerned with prediction; the physics layer is deterministic and interpretable; verification is concerned with comparison to future observations; and the portal is concerned with presentation and workflow.

The backend uses FastAPI with modular routers. The wind router contains endpoints for one-hour, three-hour, TFT three-hour and hybrid three-hour prediction. The pilot router contains flight-plan, document and weather-map functions. SQLAlchemy models define persistent structures for weather observations, forecasts, verified forecasts, users and flight plans.

The architecture also contains an automated scheduler. The project scheduler is configured to trigger monthly retraining, fetch live METAR data every 32 minutes, perform a daily 48-hour backup fetch, clean old logs and clean temporary map files. These functions support operational continuity and data maintenance.

4.3 Design alternatives and decisions

4.3.1 Linear versus cyclic wind direction

A direct linear representation of direction was rejected because it creates an artificial discontinuity at 0/360 degrees. Sine/cosine encoding was selected because it preserves circular continuity and can be decoded using atan2.

4.3.2 Tabular versus sequence modelling

XGBoost was selected because the engineered dataset contains structured meteorological and temporal features and tree-based models can capture nonlinear relationships efficiently. TFT was added because a sequence model can learn temporal relationships directly from an encoder window. The two approaches are therefore complementary rather than mutually exclusive.

4.3.3 Raw wind versus runway-relative output

Raw wind speed and direction remain important for transparency, but runway-relative values are more directly interpretable for aviation operations. The solution therefore retains the raw prediction while adding headwind and crosswind outputs.

4.3.4 Offline evaluation versus continuous verification

Offline testing is necessary for model comparison, but it cannot fully describe live performance. The system therefore stores forecast target times and supports

subsequent verification against METAR observations. This provides a path toward continuous performance monitoring.

4.4 Detailed design

The detailed design begins with the observation record. Each record contains a timestamp and meteorological fields. The feature pipeline constructs the variables required by the selected model. During inference, the most recent observation window is prepared using the same transformations used during training.

The prediction result contains wind speed and direction. Direction can be reconstructed from sine and cosine outputs. The wind vector is then passed to the runway calculation using the current runway headings. The system calculates the crosswind magnitude and signed headwind component and applies alert logic.

The pilot portal uses flight details to build a contextual weather briefing. The pilot router can determine forecast hours from flight duration, retrieve live map products and TAF information, generate a briefing PDF, and store a pilot flight-plan record. This provides an operational context for the weather information rather than displaying a generic forecast.

Figure 4.1 Individual wind component architecture.

Figure 4.2 Runway-oriented wind vector decomposition.

4.5 Standards and professional practice

The project follows aviation weather terminology such as METAR, TAF, runway heading, headwind and crosswind. Statistical reporting uses standard regression measures. The thesis uses IEEE-style references, consistent with the individual thesis template and the IT4010 formatting guidance.

Professional practice also requires separating model performance from operational certification. An MAE of approximately one knot demonstrates predictive performance on the evaluated dataset, but it does not by itself certify an aviation service. The final system should therefore be described as a research prototype and

decision-support artifact unless it undergoes the required external operational validation and approval.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Technology	Use in the individual/wider system	Reason
Python	Data processing, model training and backend support	Strong scientific and ML ecosystem.
Pandas	Data loading, cleaning and feature preparation	Efficient tabular data processing.
NumPy	Mathematical and vector calculations	Supports trigonometric wind transformations.
Scikit-learn	Metrics and model selection utilities	Standard evaluation and cross-validation tools.
XGBoost	Three-hour wind prediction	Nonlinear gradient-boosting regression.
PyTorch Forecasting	TFT sequence model	Temporal deep-learning implementation.
FastAPI	Prediction and pilot backend	Modular REST API architecture.
SQLAlchemy	Database access	Structured persistence layer.
SQLite	Local meteorological database	Lightweight project database.

APScheduler	Scheduled jobs	Automates data acquisition and maintenance.
React / Vite	Dashboard and pilot-facing interface	Interactive web presentation.
ReportLab	Generated weather briefing PDFs	Programmatic document generation.

Table 5.1: Technology Stack and Its Uses

5.2 Component implementation

5.2.1 Data pipeline implementation

The wind training script reads the weather_data table from SQLite and maps database field names to the names expected by the unified pipeline. The UnifiedWeatherPipeline is then executed with a three-hour forecast horizon. The returned features and targets are written to processed_seasonal weather_data_3h.csv before model training.

The feature engineer creates cyclic wind direction, dew-point depression, hour-of-day cyclic features, three-hour wind lag, six-hour wind rolling mean, three-hour QNH lag and QNH change. These features provide a combination of current atmospheric state, periodic information and recent temporal momentum.

5.2.2 Model implementation

The XGBoost trainer separates features and target columns and uses an 80/20 split. GridSearchCV evaluates a defined hyperparameter grid using three-fold cross-validation and negative MAE. The selected model is saved in JSON format for later inference.

The TFT training implementation creates a TimeSeriesDataSet and uses an encoder length of 48 and prediction length of three. The model uses MAE loss, early stopping and gradient clipping. The training script records the best checkpoint and validation loss in a JSON result file.

5.2.3 Backend implementation

The wind router loads the available wind models and exposes separate endpoints for one-hour, three-hour, TFT three-hour and hybrid three-hour prediction. Each endpoint prepares the current observation window, performs inference, calculates runway-relative wind, determines alert status and returns a structured response.

5.2.4 Pilot portal implementation

The pilot router accepts a flight plan containing departure, destination, departure time, area, flight levels, selected maps, pilot reference and optional flight number. It can calculate the forecast hour from flight duration, retrieve live aviation weather map products, fetch TAF information, generate a briefing PDF and store the flight plan in the database.

The database contains PilotFlightPlan records with pilot reference, flight number, departure, destination, departure time, duration and creation time. FlightTimeAndFlights provides supporting flight information such as flight number, destination, departure time and duration. RouteAlternatives provides route-level airport alternatives for selected regions.

5.3 Integration with the other components

The wind component is integrated with the common weather database and backend service. This allows the wind prediction to use current observations and allows its output to be consumed alongside other project weather components. The database also contains a common ModelsForecast structure for predicted values and a VerifiedForecast structure for later observations and derived runway values.

The project uses a modular router architecture so that wind, temperature/pressure, QNH/dewpoint and cloud/visibility functionality can be maintained separately. This is particularly important in a group project because each individual contribution can evolve without requiring all components to be rewritten.

The pilot portal acts as a user-level integration layer. Rather than exposing model endpoints directly to pilots, the portal organizes the information around a flight plan and provides supporting weather products. This is a product-level integration decision rather than a modelling decision.

5.4 Verification and software testing

5.4.1 Data tests

- Check that required weather columns exist before training.
- Check that timestamps can be reconstructed from the database fields.
- Check that chronological sorting occurs before temporal feature generation.
- Check that rows with unavailable lag/rolling values are not used for final training.
- Check that target columns are separated from the model feature matrix.

5.4.2 Model tests

- Evaluate model performance on unseen test observations.
- Compare MAE, RMSE and R^2 rather than relying on a single accuracy number.
- Check that saved models can be reloaded for inference.
- Check that model feature names align with inference features.

5.4.3 Physics and transformation tests

- Test wind-vector decomposition using known runway headings.
- Test that crosswind magnitude is non-negative in the returned aviation value.
- Test that runway selection compares headwind values.
- Test threshold behaviour above 15 and 25 knots.
- Test direction reconstruction around the 0/360-degree boundary.

5.4.4 Portal tests

- Submit a flight plan with valid departure and destination codes.
- Retrieve flight information by destination.
- Retrieve the next flight record.
- Generate a weather briefing document.
- Retrieve stored pilot documents.
- Confirm that temporary map files are cleaned according to the scheduled maintenance process.

5.5 Deployment and operation

The system is designed for continuous operation rather than one-time model execution. The project scheduler triggers live METAR retrieval every 32 minutes, performs a daily 48-hour backup retrieval, and triggers monthly retraining on the first

day of the month. The scheduler also removes old logs and temporary maps to manage storage.

The MLOps wind retrainer compares a newly trained model with the existing model using MAE before accepting the new model. This creates a basic model-governance mechanism: a new model should not automatically replace the previous model if its evaluation is worse. Earlier project logs also contain examples where newly trained models were rejected when their MAE was higher than the existing model.

The operational architecture therefore contains three monitoring layers: data freshness, model performance and system logs. This is important because a machine-learning service can fail even when its code is running if the input data are stale or the newly trained model is degraded.

Figure 5.1 Wind forecasting and verification workflow.

6 RESULTS AND VALIDATION

6.1 Research results

The final individual XGBoost evaluation uses 10,235 unseen observations. The reported test performance is MAE 1.0516 knots, RMSE 1.4404 knots and R^2 0.7798. These values should be interpreted together: MAE provides the average error magnitude, RMSE highlights larger deviations, and R^2 describes explained variance.

Metric	Final XGBoost result	Interpretation
Test observations	10,235	Unseen observations used for final evaluation.
MAE	1.0516 kt	Average absolute wind-speed error is approximately one knot.
RMSE	1.4404 kt	Larger errors increase the value above MAE.

R^2	0.7798	Approximately 77.98% of target variance is explained on this evaluation set.

Table 6.1: Final Individual XGBoost Test Performance

Figure 6.1 Final individual XGBoost wind-speed test error.

The result is stronger than the approximately 1.69-knot baseline reported in the wider project paper, but the two values should not be treated as a formal improvement claim unless the same dataset version, preprocessing pipeline and evaluation protocol are used. The individual result is reported here because it corresponds to the final evaluation values available for this thesis.

6.1.1 Model performance interpretation

An MAE of 1.0516 knots means that the typical absolute deviation between predicted and actual wind speed is close to one knot over the test sample. For an airport-level short-term forecasting system, this is a practically useful level of numerical precision. The RMSE of 1.4404 knots indicates that some observations have larger errors than the average, but the larger-error contribution remains limited relative to the operational reference values discussed in the project.

The R^2 value of 0.7798 indicates that the model captures a substantial part of the variability in wind speed. It does not mean that the model is 77.98% correct in every prediction. R^2 is a variance-explanation measure and should not be presented as classification accuracy.

6.1.2 Wind-direction results

Measure	Result
Evaluation sample	10,235 observations
Representation	Sine/cosine cyclic encoding

Evaluation metric	Circular angular MAE
Angular MAE	10.61 degrees

Table 6.2: Wind-Direction Evaluation Results

The angular MAE of 10.61 degrees indicates that the direction reconstruction has a relatively small average angular deviation. The result should be interpreted carefully because direction errors do not have the same operational effect at all angles. A ten-degree error when the wind is nearly aligned with a runway can have a different crosswind consequence from a ten-degree error when the wind is close to perpendicular to the runway.

The use of cyclic encoding is therefore important not only for numerical stability but also for the physical interpretation of the forecast. It prevents the model from treating 359 degrees and 1 degree as far apart values.

6.1.3 Runway-oriented wind results

Runway-relative quantity	MAE (kt)	RMSE (kt)
Headwind	0.8833	1.2025
Crosswind	1.2027	1.7270

Table 6.3: Runway-Oriented Wind Performance

The runway-relative results were evaluated using the final XGBoost test observations and the configured Runway 04 heading of 040 degrees. The headwind component has a lower error than crosswind. This is reasonable because crosswind is highly sensitive to directional error: when the wind vector approaches perpendicular to the runway, a small directional deviation can change the crosswind component more noticeably.

The crosswind MAE of 1.2027 knots indicates that the runway-oriented transformation remains close to the underlying wind prediction error. The RMSE of

1.7270 knots indicates that some cases have larger deviations, reinforcing the need to inspect the distribution of errors rather than relying only on an average.

6.1.4 Alert-threshold results

Threshold	Configured behaviour	Observed in test set
>15 kt	High crosswind warning	No positive cases
>25 kt	Severe crosswind status	No positive cases

Table 6.4: Crosswind Alert-Threshold Validation

No test observations exceeded the configured alert thresholds. This means that the software did not produce a false warning for the evaluated sample. However, because there were no positive hazardous cases, the dataset cannot demonstrate sensitivity, recall or missed-event performance for severe crosswinds. This is an important limitation and should be stated explicitly in any final presentation.

6.1.5 Project-level TFT and hybrid evidence

The project IEEE paper reports approximately 1.00-knot MAE for the TFT route and below 1.00-knot MAE for the hybrid wind route. The supplied verification text contains individual forecast-versus-actual examples for 1H, 3H, TFT and hybrid predictions. However, the verification export contains duplicate timestamps and pending live-data rows. Preliminary calculations from that file should therefore not replace the official evaluation script results.

For the final thesis, the safest reporting practice is to present the exact individual XGBoost metrics as the primary quantitative experiment and report TFT/hybrid values as project-level supporting results unless a clean paired evaluation export is produced. This prevents the thesis from presenting a derived or duplicated verification value as an official model score.

6.2 Product and technical results

The technical implementation provides separate API routes for one-hour, three-hour, TFT three-hour and hybrid three-hour wind prediction. The prediction response

includes predicted wind values and runway-oriented quantities. The backend also provides status messages based on crosswind thresholds.

The pilot portal provides a flight-plan workflow in which the user specifies departure, destination, departure time, flight area, flight levels, map products, pilot reference and optional flight number. The backend can determine a forecast hour from flight duration, retrieve aviation weather maps and TAF information, generate a briefing document and store the flight plan.

The database design supports this workflow through WeatherData, ModelsForecast, VerifiedForecast, User, RouteAlternatives, FlightTimeAndFlights and PilotFlightPlan structures. This is a stronger implementation than a simple model demo because the prediction is connected to persistence, verification and user workflow.

6.3 User or stakeholder validation

The supplied project artefacts demonstrate technical workflow validation but do not provide a formal sample-based user study with participant counts, questionnaire scores or statistical usability analysis. Therefore, this section does not invent a user-satisfaction percentage. Instead, validation is described at the stakeholder-workflow level.

The pilot workflow is validated by its ability to accept flight details, identify supporting flight information, retrieve aviation weather products, create a briefing document and retain the resulting flight plan. From a stakeholder perspective, these functions reduce the number of separate steps required to move from flight information to a weather briefing.

A future formal validation study could involve pilots or aviation meteorological personnel and evaluate usability, clarity of crosswind information, trust in forecast verification, response time and perceived usefulness. Such a study should be conducted under the appropriate institutional and operational approvals.

6.4 Integration results

The wind component integrates with the wider project database and FastAPI backend. Forecast and verified forecast records create a link between prediction and

observation. The scheduler provides automated data acquisition and retraining support. The pilot portal consumes the output through a user-oriented workflow.

The modular design also reduces the risk that a failure in one weather component automatically breaks all other components. The project paper describes a plugin-style architecture in which different weather modules can be maintained separately. This supports the individual nature of the research while preserving integration with the complete system.

6.5 Summary of findings

- The final individual XGBoost model achieved 1.0516-knot MAE, 1.4404-knot RMSE and 0.7798 R^2 on 10,235 test observations.
- Wind-direction angular MAE was 10.61 degrees.
- Headwind MAE/RMSE were 0.8833/1.2025 knots.
- Crosswind MAE/RMSE were 1.2027/1.7270 knots.
- No evaluated test observation exceeded the configured 15-knot or 25-knot crosswind thresholds.
- The backend supports one-hour, three-hour, TFT and hybrid wind prediction routes.
- The pilot portal supports flight planning, route information, weather briefing and aviation weather map integration.
- The current evidence supports technical validation but not a formal human-subject usability claim.

7 DISCUSSION

7.1 Interpretation of findings

The final XGBoost result demonstrates that airport-level wind prediction is feasible using a structured machine-learning pipeline. The average absolute error is close to one knot, and the R^2 value indicates substantial explanatory power. The result is particularly relevant because the input data are localized airport observations rather than a broad regional grid.

The wind-direction result reinforces the importance of using a representation that respects the circular nature of the variable. An angular MAE of 10.61 degrees is

more meaningful than a conventional linear degree error because it correctly handles the boundary between 359 degrees and 1 degree.

The runway-relative results demonstrate that prediction performance can be translated into operationally meaningful quantities. Headwind and crosswind are not independent models; they are deterministic transformations of predicted wind speed and direction. Their errors therefore provide a useful measure of how forecast uncertainty propagates into aviation interpretation.

The lower headwind error compared with crosswind error is important. Headwind depends on the cosine of the relative angle, while crosswind depends on the sine. Around a runway-aligned wind, crosswind is small and highly sensitive to directional changes; around a perpendicular wind, crosswind becomes dominant. This means that the same directional prediction error can have different operational consequences depending on the current wind geometry.

7.2 Comparison with literature baselines and existing solutions

The wider project paper reports a baseline XGBoost wind MAE of approximately 1.69 knots, approximately 1.00-knot TFT performance and a hybrid result below one knot. The individual final XGBoost result of 1.0516 knots is numerically lower than the earlier project baseline. However, the evaluation contexts must be distinguished because changes in dataset version, preprocessing, feature engineering, target construction or train-test split can affect MAE.

The individual research also extends beyond pure model comparison. The literature sources describe machine learning for atmospheric prediction and aviation weather support, but the individual component combines prediction with runway-oriented physical interpretation and a pilot portal. The contribution is therefore best understood as an integrated decision-support pathway rather than a new ML algorithm.

The use of continuous forecast verification also distinguishes the solution from a static model experiment. By storing target times and later observations, the system creates the foundation for tracking whether model performance remains stable as atmospheric conditions change.

7.3 Research contribution

The primary research contribution is the design and evaluation of an airport-level wind forecasting pipeline specifically structured for aviation interpretation. The work demonstrates a complete sequence from local observations to machine learning prediction, circular direction handling, runway-vector decomposition and verification.

A second contribution is methodological clarity around direction. The use of sine/cosine encoding and angular error avoids a common modelling problem in directional variables. A third contribution is the explicit separation between statistical prediction and deterministic aviation transformation, making the system easier to explain and audit.

A fourth contribution is the use of forecast verification as part of the architecture rather than as a one-time academic calculation. This creates a pathway for MLOps monitoring and future model retraining.

7.4 Product contribution

The product contribution is the pilot portal. The portal turns a prediction service into a user workflow. The pilot enters flight information, the backend can identify supporting flight information and forecast duration, and the system can generate weather briefing material and map products. This provides context for the wind forecast and reduces the need for a user to interpret raw API responses.

The product also creates a persistent record of flight plans and generated documents. This supports traceability and provides a foundation for later auditing or historical review.

7.5 Limitations and threats to validity

- Random 80/20 splitting is convenient and matches the implemented trainer, but a chronological or rolling-origin split would be more appropriate for strict future forecasting validation.
- The final XGBoost metrics should not be directly compared with other project metrics unless the evaluation dataset and preprocessing are identical.

- The supplied verification export contains duplicate timestamps and pending live-data rows, so it should be cleaned before being used as an official continuous-verification dataset.
- The test sample contained no hazardous crosswind cases above the implemented thresholds; therefore alert sensitivity cannot be established.
- Airport-specific models may not generalize to other airports because local terrain, runway orientation and atmospheric patterns differ.
- The model does not provide a formal probabilistic uncertainty interval in the current implementation.
- The portal has technical workflow validation but no formal statistically analysed human-participant usability study in the supplied evidence.
- Operational aviation use would require appropriate certification, governance and validation beyond the scope of this academic prototype.

7.6 Implications

The research suggests that hyper-local airport observations can support useful short-horizon wind prediction when combined with temporal feature engineering. More importantly, the study demonstrates that prediction accuracy can be connected to aviation interpretation without requiring a second machine-learning model for runway components. Deterministic vector mathematics provides an explainable bridge from predicted weather to operational indicators.

For future airport deployments, the same architecture could be reused by changing the historical dataset, runway configuration and local validation procedure. This creates a scalable pattern for airport-specific decision-support systems while retaining local model training and verification.

The MLOps architecture also has implications for long-term reliability. Because weather behaviour and data distributions can change, model performance should be monitored over time rather than assuming that one training run remains optimal indefinitely.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The primary value proposition is to provide airport-specific short-term wind information in a form that is easier to interpret for aviation operations. The target users include pilots, flight operations personnel and aviation meteorological stakeholders. The system does not attempt to compete with authoritative aviation weather services; instead, it can operate as a localized analytics and decision-support layer.

For a pilot, the value is the reduction of manual interpretation. The system can present predicted wind, runway-relative components, crosswind status and supporting weather information within a flight-planning workflow. For an aviation weather stakeholder, the value is the ability to compare model predictions with later observations and monitor model behaviour.

8.2 Market and competitive position

The solution is positioned between general weather applications and specialist aviation meteorological systems. General weather applications may provide broad conditions but are not necessarily tailored to runway geometry. Specialist aviation systems may provide authoritative products but can be costly or difficult to customize for research-specific airport-level models.

The differentiating feature of this research is the integration of hyper-local ML prediction, runway-relative wind interpretation, continuous verification and pilot-facing briefing functions. The competitive advantage is therefore architectural and workflow-based rather than the invention of a new machine-learning algorithm.

8.3 Business or sustainability model

A future deployment could use an institutional licensing model for airports or aviation organizations, a software-as-a-service model for flight operators, or a research deployment model in which airport-specific forecasting modules are maintained centrally. The exact commercial model would depend on regulatory requirements, data licensing and the responsibilities of the operational organization.

Sustainability depends on automated observation ingestion, scheduled model retraining, monitoring of model error and maintenance of the pilot portal. The existing scheduler and MLOps components provide a technical foundation for these activities.

8.4 Scalability intellectual property and compliance

The modular architecture can be extended to other airports by creating airport-specific datasets, runway configurations and validation procedures. The core software architecture can remain reusable. However, scaling should not mean copying one model without retraining; local atmospheric behaviour must be evaluated at each new site.

Software code, trained models and documentation may form intellectual-property assets subject to institutional and project agreements. Any operational deployment would also need to address data licensing, authentication, privacy, aviation safety governance and any applicable certification or approval process.

8.5 Contribution to the Sustainable Development Goals

The research has an indirect contribution to Sustainable Development Goal 9 through intelligent digital infrastructure and technology development. It can also support Goal 13 by improving the use of environmental observations and climate-related information in a practical decision-support system. The direct project objective remains aviation safety, operational efficiency and improved use of localized meteorological data.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This individual thesis developed and evaluated a machine learning-based hyper-local wind prediction and aviation decision-support component for Bandaranaike International Airport, under varying local weather conditions. The research used airport-level historical observations, temporal feature engineering, cyclic wind-direction representation, XGBoost and TFT forecasting, runway vector decomposition and a pilot portal.

The final individual XGBoost evaluation produced an MAE of 1.0516 knots, RMSE of 1.4404 knots and R^2 of 0.7798 on 10,235 test observations. Wind-direction evaluation produced an angular MAE of 10.61 degrees. The predicted wind was converted into runway-oriented values, with Runway 04 headwind MAE/RMSE of 0.8833/1.2025 knots and crosswind MAE/RMSE of 1.2027/1.7270 knots.

The software contribution extends beyond prediction. The backend provides separate wind forecasting routes, deterministic runway calculations, alert logic, forecast storage and verification support. The pilot portal provides flight planning, route information, weather briefing generation and aviation map integration. Together these components create a complete pathway from airport observations to operationally interpretable decision support.

The research also identifies important limitations. The random train-test split used by the implemented trainer is not ideal for strict future forecasting evaluation. Rare hazardous crosswind events are absent from the test sample. The project-level TFT and hybrid metrics require clean paired evaluation before being treated as final individual results. Finally, the prototype is not an operationally certified aviation service. These limitations define clear directions for future work.

9.2 Achievement of objectives

Objective	Achievement	Evidence
O1: Collect/analyse historical VCBI wind data	Achieved	2019–2026 project database and wind-focused preprocessing.
O2: Identify temporal patterns / engineer features	Achieved	Lag, rolling, hour-of-day and cyclic direction features.
O3: Develop/evaluate ML wind models	Achieved	XGBoost and TFT implementations.
O4: Handle/evaluate circular direction	Achieved	Sine/cosine encoding and 10.61° angular MAE.

O5: Convert wind to runway components	Achieved	Runway 04/22 vector calculations and evaluated HW/CW metrics.
O6: Implement crosswind alert logic	Achieved	15-knot warning and 25-knot severe thresholds.
O7: Support forecast verification	Achieved	Forecast target timestamps and VerifiedForecast database structure.
O8: Develop pilot portal workflow	Achieved	Flight plan, briefing, maps, route information and document retrieval.

Table 9.1: Individual Research Objective Achievement Summary

9.3 Individual research and product contributions

The individual research contribution is the wind-focused scientific pipeline: historical wind analysis, feature engineering, XGBoost/TFT forecasting, circular direction handling, evaluation, runway-oriented transformation and verification. The individual product contribution is the pilot-facing workflow that contextualizes the forecast using flight information and weather briefing functions.

The wider project infrastructure is acknowledged as shared. This distinction allows the thesis to remain an individual research document while still explaining the integration points required for the complete system.

9.4 Future directions

6. Replace the random 80/20 split with chronological and rolling-origin evaluation to measure true future generalization.
7. Recalculate TFT and hybrid performance using a clean, deduplicated and paired forecast-versus-actual evaluation set.
8. Expand the test set to include rare high-wind and hazardous crosswind cases.

9. Evaluate performance separately across local weather conditions phases and inter-varying weather conditions.
10. Introduce prediction intervals or probabilistic forecasts so that uncertainty is communicated to users.
11. Investigate calibration and confidence scoring for runway-relative wind conditions.
12. Evaluate the architecture at additional Sri Lankan airports with airport-specific retraining.
13. Conduct a formal usability study with appropriate aviation stakeholders if institutional and operational approvals are obtained.
14. Integrate stronger model-governance rules and maintain model-version history, data-version history and acceptance criteria.
15. Add more explicit data-freshness indicators and verification history to the pilot portal.
16. Evaluate whether external meteorological and remote-sensing variables can improve short-horizon wind prediction without introducing data leakage.
17. Before operational use, complete the required aviation-domain validation, safety assessment and governance process.

REFERENCES

[1] A. P. Wickramagamage, “seasonal weather Rainfall Forecasting in Sri Lanka Using Artificial Neural Networks,” 2011.

[2] S. K. Jayawardena, “Machine Learning Based Weather Prediction Model for Short Term Weather Prediction in Sri Lanka,” 2021.

[3] Z. Wang, “Analysis of the Most Common Aviation Weather Hazard and Its Key Mechanisms over the Yangon Flight Information Region,” Journal of Meteorology, 2022.

[4] R. Jayasinghe, “Climatology of Lightning Activities over Bandaranaike International Airport in Sri Lanka,” Atmospheric Research Study.

[5] H. W. and M. Sprenger, “A Review of High Impact Weather for Aviation Meteorology,” Pure and Applied Geophysics, Springer, 2019.

[6] S. R. Jeong, “AI Framework for Automated Terminal Aerodrome Forecasting,” Aviation, 2024.

[7] X. Shi et al., “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting,” Advances in Neural Information Processing Systems, 2015.

[8] M. Chen, “Deep Learning for Weather Forecasting: A Review,” Atmospheric Research, 2020.

[9] P. Bauer, A. Thorpe and G. Brunet, “Challenges and Design Choices for Global Weather and Climate Models Based on Machine Learning,” Geoscientific Model Development, vol. 11, 2018.

[10] Y. Rasp and S. Lerch, “Neural Networks for Postprocessing Ensemble Weather Forecasts,” Monthly Weather Review, vol. 146, no. 11, pp. 3885–3900, 2018.

[11] “Hyper-Local Aviation Weather Forecasting and Verification for Sri Lankan Monsoon Conditions,” IEEE-style project research paper supplied for this study.

[12] R. A. A. D. Ranasingha, “Research Proposal IT22269720 R26-IT-132: AI-Based Hyper-Local Aviation Weather Forecast Verification System for Sri Lankan Monsoon Patterns,” SLIIT, 2026.

GLOSSARY

Term	Definition
Airport-level / hyper-local forecasting	Forecasting focused on atmospheric conditions at a specific airport rather than a broad region.
Crosswind	Wind component perpendicular to the runway heading.
Headwind	Wind component aligned opposite the aircraft's direction of travel along the runway, supporting take-off/landing performance.
METAR	Standardized meteorological observation report for an aerodrome.
TAF	Terminal Aerodrome Forecast covering expected conditions for an airport over a defined period.
Wind vector	A representation of wind magnitude and direction.
Cyclic encoding	A mathematical representation that preserves the circular nature of a variable.
Forecast target time	The future timestamp for which a prediction is generated.
Verification	Comparison of a prediction with an observation available for the same target time.

MLOps	Practices for maintaining, monitoring and updating machine-learning systems in operation.
Temporal Fusion Transformer	A neural forecasting architecture designed for temporal and multi-variable sequence modelling.
Runway heading	The direction, in degrees, associated with the runway orientation used for vector calculations.

Table A.1: Glossary of Technical Terms

APPENDICES

Appendix A: Research instruments or extended results

This appendix contains extended evidence that is useful for understanding the individual component but would interrupt the flow of the main thesis if presented in full. The final submission can include selected prediction logs, detailed model outputs, additional error distributions, screenshots and test cases here.

A.1 Extended evaluation summary

Measure	Value	Comment
Historical database	121,407 records	2019–2026 project database.
Cleaned 3H data	51,175 rows	Model-ready data after preprocessing.
XGBoost train/test	40,940 / 10,235	80/20 implementation split.
XGBoost MAE	1.0516 kt	Final individual test result.
XGBoost RMSE	1.4404 kt	Final individual test result.
XGBoost R^2	0.7798	Final individual test result.
Direction angular MAE	10.61°	Circular direction evaluation.
Headwind MAE/RMSE	0.8833 / 1.2025 kt	Runway 04 evaluation.
Crosswind MAE/RMSE	1.2027 / 1.7270 kt	Runway 04 evaluation.
Alert positives	0 in evaluated test set	No test cases above 15/25 kt thresholds.

Table A.2: Additional Evaluation Measures and Interpretation

A.2 Verification-data handling note

The supplied verification export contains rows with forecast type labels such as 1H, 3H, TFT_1H, TFT_3H and HYBRID_3H. It also contains pending live-data rows and repeated timestamps. Before using this export as an official thesis table, the records should be deduplicated by target time and forecast type, pending rows should be excluded, and the resulting paired predictions should be re-evaluated using one consistent metric implementation.

A.3 Suggested final figures

The final thesis should insert a predicted-versus-actual wind-speed line/scatter plot, a wind-direction error distribution, a histogram of absolute wind-speed errors, and a runway headwind/crosswind comparison. These figures should be generated directly from the official final evaluation CSV so that every plotted value is traceable to the reported metrics.

A.4 Pilot portal evidence

Insert the actual pilot dashboard screenshot from the final implementation here. The screenshot should be captioned and referred to in Chapter 6. The recommended evidence includes: login or authenticated pilot area, flight-plan form, route information, weather briefing output, wind information, map products and generated-document history.

A.5 Individual-component boundary and responsibility

The individual contribution documented in this thesis is the wind-prediction and aviation interpretation pathway, rather than a claim of sole authorship of the complete group platform. The implementation evidence shows a wind model pathway, runway-relative calculations, alert logic and pilot-facing backend routes. Other environmental variables and the overall platform are described only where they provide necessary context for integration.

This boundary matters for academic attribution. A group system may contain multiple models, data acquisition services, a database, frontend pages and operational utilities. Their presence in the repository does not by itself establish that every part was designed or implemented individually by the author. The thesis therefore distinguishes the assigned wind component from shared infrastructure. Claims about individual contribution should be supported by the allocated topic assessment form, source files, version history, meeting records or supervisor confirmation where available.

The individual component can be described as a connected chain: prepare historical observations; create time-related and wind-related features; train and evaluate prediction models; expose predictions through backend routes; convert wind forecasts into runway-relative components; and present outputs to the intended pilot-facing workflow. This chain provides a coherent scope without taking credit for unrelated system modules.

For final submission, any sentence using “I developed” should refer only to work supported by the individual allocation or direct implementation evidence. Shared work should use wording such as “the wider project integrates” or “the system includes”. This distinction improves traceability and makes the viva explanation more defensible.

A.6 Evidence register and claim status

An evidence register links thesis claims to the artefact that supports them. The current project materials support several different levels of claim, and those levels should not be treated as interchangeable. Source code demonstrates that a calculation or endpoint is implemented. A saved model-evaluation result supports a performance claim only for the dataset and split used to generate it. A research paper reports project-level results, but does not automatically prove that the same values came from the individual component's final evaluation run.

The recorded database count is 121,407 observations. The unified three-hour pipeline has been reported to retain 51,175 clean rows. The XGBoost trainer uses an 80/20 split with random_state=42, and the associated split sizes are 40,940 training records and 10,235 test records. The recorded individual XGBoost metrics are MAE 1.0516 knots, RMSE 1.4404 knots and R^2 0.7798. The wind-direction angular MAE is recorded as 10.61 degrees; runway-relative headwind MAE is 0.8833 knots and crosswind MAE is 1.2027 knots.

The project paper separately reports approximately 1.00-knot MAE for TFT and below 1.00-knot MAE for the hybrid route. Those values must be labelled as paper-reported project results unless the final saved evaluation output confirms the exact run, split and sample count. The supplied verification export contains pending observations and duplicate-looking entries; preliminary means from it should not replace a clean official evaluation.

A final evidence register should record claim, evidence file, date/version, evaluation subset and status. Where evidence is absent, the thesis should say "not measured in the available artefacts" rather than supplying an assumed value.

A.7 Data source and observation meaning

The weather data used by the project represent observations for the VCBI airport context. The repository and project paper describe historical METAR-based data and a live METAR acquisition path. METAR is an observation report, not a forecast product. This distinction is important because observed values are used to describe weather conditions at an observation time, while model outputs estimate a target time in the future.

The database schema includes year, month, date, UTC time, visibility, weather, clouds, wind direction, wind speed in knots, dry-bulb temperature in Celsius, dew point in Celsius, relative humidity and QNH in hectopascals. The presence of these columns does not mean that every variable is an output of the individual wind model. In the wind component, wind speed is the principal predicted quantity in the reported XGBoost evaluation; wind direction and temporal variables can also be used to construct features or support downstream calculations.

A defensible dataset description states the station, date coverage, record count, units and the rules used to interpret missing or invalid observations. It should not claim that every observation is independent: reports may be temporally adjacent, and repeated or revised reports may occur. The record count alone also does not establish that all rows are complete or suitable for training.

The available project notes describe coverage as 2019–2026, while a supplied Excel workbook is named for 2019–2024. These are different artefacts and should not be silently conflated. The thesis uses the final database and pipeline counts as reported project evidence, and flags the Excel date range as a separate supplied source requiring reconciliation if it is used as the definitive dataset citation.

A.8 Data dictionary: temporal fields

The temporal fields are essential because weather observations have meaning only when associated with a valid time. The database schema stores year, month, date and UTC time. A pipeline may combine these components into a timestamp for sorting, lag construction and forecast-target alignment. UTC is useful because it avoids ambiguity caused by local time-zone changes and allows consistent matching between historical records and target times.

Before creating lags, timestamps should be parsed consistently and records should be sorted chronologically within the relevant station or series. If a timestamp cannot be parsed, the row cannot safely participate in time-dependent feature creation until the issue is resolved. If two records share a timestamp, the data-cleaning policy should determine whether they are true duplicates, repeated observations or distinct report types. Blindly deleting all equal timestamps can discard legitimate information; retaining all duplicates can distort rolling features.

The feature engineer in the project creates hour-related fields, including Hour, Hour_Sin and Hour_Cos. Cyclic hour encoding represents the daily cycle without treating midnight and 23:00 as far apart. The project also creates lagged wind-speed features and rolling summaries. These derived columns depend on the timestamp order and must be created after ordering and cleaning.

The thesis should preserve the original observation timestamp separately from any derived forecast target time. This separation allows an evaluator to answer two different questions: when was the information available to the model, and what future time did the model attempt to predict? Confusing these times can create leakage and make an apparently accurate evaluation invalid.

A.9 Data dictionary: wind variables and units

Wind speed is recorded in knots in the project database and reported model metrics. One knot is one nautical mile per hour. Keeping the unit consistent is important because the same numeric value means something different in metres per second or kilometres per hour. If a conversion is ever applied, the conversion should be explicit and should occur at a clearly defined boundary in the pipeline.

Wind direction is an angular value measured in degrees. It is circular rather than linear: 359 degrees and 1 degree are only two degrees apart in the physical direction even though ordinary subtraction produces 358 degrees. This property explains why the feature engineer creates sine and cosine representations. It also explains why direction errors should be evaluated with an angular difference rather than a simple arithmetic difference.

Runway heading is also expressed as a direction angle. In the implementation, Runway 04 is represented with a heading of 40 degrees and Runway 22 with a heading of 220 degrees. These are the values used by the current runway-selection logic; the thesis does not claim that they are a complete airport operating procedure or that the system accounts for every operational factor used by airport authorities.

Wind direction convention must be stated carefully. Meteorological wind direction usually indicates the direction from which the wind comes, while vector equations may be written using direction conventions that describe where a vector points. The code currently computes the difference between wind direction and runway heading and derives magnitudes from sine and cosine. This implementation detail should be documented, and any production use should verify the convention against the authoritative aviation source and expected runway-relative sign conventions.

A.10 Data dictionary: atmospheric variables

The weather database also stores visibility, weather codes or descriptions, cloud information, dry temperature, dew point, relative humidity and QNH. These fields provide context for the wider hyper-local weather system. They are not all direct outputs of the individual wind model, so they should not be described as wind-model targets unless the specific model configuration proves that role.

Dry-bulb temperature describes the ambient air temperature. Dew point is related to the moisture content of the air. Relative humidity is stored as a percentage. QNH is an atmospheric pressure setting reported in hectopascals in this project. Visibility and cloud fields are operationally relevant weather descriptors, but their interpretation can involve categorical or coded values rather than continuous numeric measurements.

The feature engineering code includes dew-point depression, calculated as dry temperature minus dew point, and QNH-related lag/change features. These examples demonstrate how the wider pipeline can derive additional variables from observed measurements. The existence of such features does not establish that they improve wind prediction; that conclusion would require an ablation experiment comparing otherwise equivalent models with and without the features.

For reproducibility, the data dictionary should record each raw field, its unit, type, missing-value convention, whether it is used by the wind pathway, and whether it is a derived field. If an input is only used by another component, mark it as contextual rather than implying it is part of the individual model's feature set.

A.11 Cleaning decisions and their consequences

Cleaning is not merely a cosmetic stage. Every cleaning rule changes the examples available for training and can affect the distribution of weather conditions represented in the final data. The project pipeline performs timestamp ordering, duplicate handling and missing-value treatment before model training. The unified three-hour pipeline is reported to retain 51,175 rows from the larger historical database.

Rows may be excluded because a required target is missing, because lag or rolling calculations cannot be formed at the beginning of a sequence, or because source values are invalid. A lag feature necessarily requires earlier observations; a rolling mean requires a sufficient history. Consequently, the processed row count can be smaller than the raw count even when the source database is not corrupted. The 51,175 count should be described as the retained output of the specified three-hour pipeline, not as a universal count for every model or horizon.

The order of operations matters. If rolling statistics are calculated before sorting, they may represent the wrong time direction. If missing target rows are imputed without a documented policy, the target may no longer represent an actual observation. If train and test rows are randomly split after highly overlapping lag windows are created, neighbouring observations can appear in both sets and make the test set less independent.

A robust cleaning report records counts at each stage: raw input, timestamp-valid rows, duplicate treatment, rows with required variables, rows lost to feature warm-up, and final training-ready rows. The available materials provide the raw and final pipeline counts, but not a complete stage-by-stage audit table. Such intermediate counts should be generated from the final run rather than invented for this thesis.

A.12 Lag features and rolling summaries

A lag feature places an earlier observation beside the current prediction example. The project feature engineer includes a three-hour lag for wind speed and a six-hour rolling mean. The purpose is to give the model information about recent conditions, since short-term weather evolution is often related to the immediately preceding state.

A three-hour lag should be interpreted relative to the dataset's sampling interval. If the input observations are exactly hourly, a shift of three rows corresponds to three hours. If intervals are irregular or reports are missing, a three-row shift is not guaranteed to equal three elapsed hours. Therefore, the pipeline must either regularize the time series or document the exact meaning of its row-based shifts. The same caution applies to rolling windows: six records and six elapsed hours are not necessarily equivalent when data gaps exist.

The rolling mean smooths short-term fluctuations, but smoothing can also hide abrupt changes. For aviation use, this is a trade-off rather than an automatic improvement. A rolling average may help the model identify the recent background state while a separate current observation captures a rapid change. The value of these features should be assessed through out-of-sample evaluation, not assumed from their intuitive appeal.

Feature creation must avoid looking into the future. At prediction time, the input features should be calculated only from observations that would have been available then. A shift in the wrong direction or a centered rolling window can inadvertently include future values. The implementation should therefore use past-directed shifts and rolling windows, and a unit test should confirm that changing a future observation does not alter earlier feature rows.

A.13 Cyclic encoding of wind direction

The feature engineer converts wind direction degrees to sine and cosine values. For an angle θ in degrees, the two derived values are sin(θ) and cos(θ), after conversion to radians. This creates a two-dimensional representation of direction on the unit circle. It avoids the discontinuity that occurs at the 0/360-degree boundary.

For example, directions near north have sine values close to zero and cosine values close to one. Directions on opposite sides of the 180-degree boundary are represented near the corresponding opposite point on the circle. The model can learn that 359 degrees and 1 degree are close because their sine/cosine pairs are close. This is more suitable than presenting the raw angle as a single linear number to a model that does not inherently understand circular geometry.

The representation does not by itself produce a wind-direction forecast. It is a feature encoding. If the model predicts wind speed while direction sine and cosine are inputs, the thesis must not claim that the model predicts direction solely because direction fields are present in the dataset. A direction prediction task would require an explicit direction output or separate component and an appropriate angular evaluation metric.

The project records a wind-direction angular MAE of 10.61 degrees in its evaluation notes. That value should be tied to the relevant direction output or reconstruction procedure when presented as a result. If the exact model and evaluation split are not documented alongside the metric, describe it as a recorded project evaluation result and avoid assigning it to an unverified training run.

A.14 Hour-of-day encoding

The feature engineer creates Hour, Hour_Sin and Hour_Cos. The raw hour indicates the hour of day, while the sine and cosine values represent the daily cycle continuously. This helps avoid a sharp numerical boundary between 23:00 and 00:00. The encoding is mathematically analogous to cyclic wind-direction encoding, but the cycle length is 24 hours rather than 360 degrees.

For hour h, a cyclic representation can be written as $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$. These features provide a periodic signal to the model. The source code confirms the presence of Hour_Sin and Hour_Cos; the equation here explains their general meaning. The final thesis should ensure that the exact implementation formula is consistent with the code version submitted for assessment.

Time-of-day features can help a model represent recurring daily patterns, but they do not prove that a daily pattern exists in the target variable or that the feature is causally responsible for better accuracy. An ablation experiment would be needed to quantify its contribution. The feature may also behave differently across seasons and weather regimes, so performance should be examined across relevant temporal subsets where enough observations exist.

A key implementation check is that hour values are derived from the same timezone convention as the underlying timestamp. Mixing local time and UTC would shift the cyclic features and could introduce a systematic mismatch between the model's learned pattern and the prediction-time inputs.

A.15 XGBoost model: methodological explanation

XGBoost is a gradient-boosted decision-tree method. It builds an ensemble of trees sequentially, with each new tree contributing to the correction of errors left by the existing ensemble. Tree-based methods can model nonlinear relationships and interactions among input features without requiring all relationships to be specified as linear equations.

The project wind trainer uses XGBoost and evaluates the resulting model using MAE, RMSE and R^2. The recorded search configuration includes 300, 500 and 1,000 estimators; maximum depths of 5, 7 and 9; learning rates of 0.01 and 0.05; subsampling values of 0.8 and 1.0; and column-sampling values of 0.8 and 1.0. GridSearchCV is configured with three folds and negative MAE scoring. These are implementation settings, not evidence that every possible hyperparameter combination was tested beyond the defined grid.

The trainer uses an 80/20 train-test split with random_state=42. This makes the split repeatable for the same input order and software behaviour, but it is not equivalent to a chronological holdout. For time-dependent weather forecasting, random splitting can place nearby observations in both train and test sets. The resulting metrics should therefore be interpreted as performance under the implemented split, with chronological testing recommended as a stronger future validation design.

The recorded XGBoost metrics are MAE 1.0516 knots, RMSE 1.4404 knots and R^2 0.7798. These values should be reported together with the target variable, split method and units. They should not be compared with other metrics unless the compared evaluations use compatible data, horizon, target and split procedures.

A.16 Hyperparameter search and reproducibility

Hyperparameter search explores a predefined set of model configurations to identify a configuration that performs well under a chosen validation procedure. In this project, the trainer uses GridSearchCV with three folds and negative mean absolute error as the scoring measure. The negative sign is used because the scikit-learn search interface treats larger scores as better, while MAE is a quantity that should be minimized.

The grid includes tree count, maximum depth, learning rate, row subsampling and column subsampling. Increasing the number of trees can improve fit but also increases training time. Greater depth allows more complex feature interactions but can increase overfitting risk. Learning rate controls the contribution of each tree; smaller values often require more trees. Subsampling introduces variation and can reduce overfitting, though it may also increase bias.

Reproducibility requires more than listing the grid. The submitted artefacts should identify the training script, dataset version, random seed, selected best parameters, software environment and saved evaluation output. If the best parameters are not present in the final logs, the thesis should report the search space and selection procedure but should not invent a winning configuration.

The current project notes preserve the search configuration and split settings, but the thesis should be cautious about claiming exact deployment parity unless the deployed model file can be linked to the training run. A model artifact, configuration file and metrics file with matching timestamps or version identifiers would make this link stronger.

A.17 Train-test splitting and temporal leakage

The XGBoost training script uses an 80/20 split and random_state=42. The recorded split sizes are 40,940 training examples and 10,235 test examples. This split is reproducible, but it has a limitation for time-series data: neighbouring observations can share weather context and overlapping lag histories. Random allocation can place some of those related examples in both the training and test subsets.

Temporal leakage occurs when information that would not have been available at the prediction time influences training or evaluation. It can happen through future-derived features, preprocessing fitted to the complete dataset, or train-test contamination caused by overlapping windows. A random split does not automatically prove that leakage occurred, but it does not provide the same protection as a chronological holdout.

A stronger future evaluation would select a cutoff time, train only on observations before that cutoff, and test on later observations. For rolling-origin evaluation, the cutoff could be moved forward and the model evaluated over multiple future windows. This would show whether the model's performance is stable across different periods. The thesis should describe this as a recommended evaluation improvement unless those experiments have actually been run.

The test metrics in this thesis must retain their original qualification: they describe the implemented split, not guaranteed future operational accuracy. This is especially important when discussing deployment or aviation decision support. A high R^2 or low average error on one split is not proof of performance during rare, rapidly changing or extreme weather conditions.

A.18 Temporal Fusion Transformer configuration

The project contains a Temporal Fusion Transformer (TFT) training route implemented with PyTorch Forecasting and PyTorch Lightning. The recorded configuration includes a maximum encoder length of 48, maximum prediction length of 3, batch size 256, learning rate 0.03, hidden size 32, two attention heads, dropout 0.15 and hidden continuous size 16. The run is configured for up to 25 epochs, MAE loss, early stopping with patience 3 and gradient clipping at 0.1.

The encoder provides the model with a history window, while the prediction length specifies the forecast sequence length used by the training setup. The code comments and configuration should be used to interpret whether those lengths correspond to hours or time steps for the actual sampling frequency. It is unsafe to equate a number of steps with a number of hours without confirming the dataset interval.

TFT combines sequence modelling with attention and gating components intended to learn temporal relationships. Its complexity can be useful where past values and time-varying inputs matter, but it also introduces additional training, tuning and data-structuring requirements compared with a tabular gradient-boosting model. A valid comparison should use the same forecast target, horizon and test period wherever possible.

The wider project paper reports TFT wind MAE of approximately 1.00 knot. Unless a final run-specific evaluation artifact establishes the exact sample count and test window, the number should be presented as a paper-reported result rather than a fully reconciled final metric for every TFT route.

A.19 Sequence dataset construction

A sequence model requires examples that preserve the order of observations. The project uses PyTorch Forecasting's TimeSeriesDataSet and a dataset-builder script for the TFT pathway. The project notes report that the TFT pipeline had 67,563 valid samples after grouping and removing invalid dates or missing steps. This figure refers to the reported TFT sequence preparation and should not be conflated with the 51,175 rows retained by the unified three-hour XGBoost pipeline; the two counts can arise from different preparation logic and should be explained separately.

Sequence construction has several important requirements. Each time series must have a consistent ordering index. Encoder and prediction windows must not cross between unrelated series. Missing time steps need a defined policy because a model that expects regular steps may interpret an unmarked gap as a normal interval. Target values and future-known inputs must be separated correctly so the model cannot use future observations as historical inputs.

The TFT pipeline uses shuffle=False in the time-series workflow to preserve sequence order at the relevant stage. This does not mean that all training frameworks or batches must always remain unshuffled; the critical requirement is that each encoded sequence represents the correct ordered time window. The thesis should describe the implemented setting rather than generalize it to every internal operation.

For reproducibility, the final project package should preserve the dataset-builder configuration, grouping key, time index definition, missing-step policy and resulting dataset counts. If any of these details change between model runs, metrics may no longer be directly comparable.

A.20 Hybrid forecast concept and interpretation

The project includes a hybrid three-hour wind route that combines XGBoost and TFT predictions. The motivation for combining the models is that the two methods represent temporal information differently: XGBoost learns from an engineered tabular feature set, while TFT uses sequence windows. Their errors may therefore differ across weather situations.

A hybrid is not automatically better than either constituent model. The benefit depends on the combination rule and on whether the model errors are complementary. The project implementation describes an average hybrid route. If the forecasts are combined by arithmetic averaging, the hybrid prediction is the mean of the constituent outputs available for the same target time. This explanation should be checked against the exact submitted code version before the equation is included as a definitive implementation specification.

A fair evaluation compares XGBoost, TFT and the hybrid on aligned target times and the same evaluation subset. It should report the number of matched examples, missing predictions, the target unit, and MAE/RMSE. If one model's output is absent for a target time, the hybrid rule needs a documented fallback or that row should be excluded consistently. Comparing metrics calculated on different subsets can make a hybrid appear better or worse for reasons unrelated to the combination method.

The project paper reports below 1.00-knot MAE for the hybrid route. The available verification export has fewer non-pending hybrid rows than the other forecast types and contains duplicate-looking entries. The export's preliminary average must therefore not be presented as a definitive final hybrid metric until the evaluation set is cleaned and reconciled.

A.21 Mean absolute error: definition and use

Mean absolute error is the average absolute difference between predictions and observed values. For n observations, MAE equals the sum of absolute prediction errors divided by n. Because absolute values are used, positive and negative errors do not cancel each other. For a wind-speed target measured in knots, MAE is also measured in knots.

MAE is relatively easy to explain operationally: it describes the average magnitude of the model's error on the evaluated examples. It does not indicate the direction of bias, and it does not show how often large errors occur. Two models can have similar MAE values but different error distributions, so a complete evaluation should also inspect RMSE, quantiles or plots when sufficient clean data are available.

The recorded individual XGBoost wind-speed MAE is 1.0516 knots. It should be interpreted with the associated test size of 10,235 records and the random 80/20 split used by the trainer. The result is not a universal bound on future errors and should not be described as "always within one knot". Some individual predictions can have much larger errors than the mean.

MAE values should be compared only when they refer to the same target and compatible test examples. A one-hour model, three-hour model, TFT sequence model and hybrid model may have different horizons or sample availability. A table comparing them should clearly identify the horizon, sample count and source of each reported value.

A.22 Root mean squared error: definition and use

Root mean squared error is calculated by squaring each prediction error, averaging the squared values, and taking the square root. RMSE is expressed in the same unit as the target. Because errors are squared before averaging, larger errors receive greater weight than they do under MAE.

The recorded XGBoost RMSE is 1.4404 knots. Its value is higher than the recorded MAE of 1.0516 knots, which is a common pattern when some errors are larger than the typical absolute error. This difference alone does not reveal the exact number of outliers or their causes; the full error distribution would be needed for that interpretation.

RMSE is useful alongside MAE because it makes the effect of larger errors more visible. However, it remains an aggregate statistic. It cannot show whether errors cluster during particular hours, weather regimes, wind directions or rapid transitions. Those questions require stratified evaluation and plots based on the clean test predictions.

For the thesis, the safest reporting format includes the metric name, numerical value, unit, model, forecast horizon, split method and sample count. If an exact test count is unavailable for a particular TFT or hybrid result, state that limitation instead of borrowing the XGBoost count.

A.23 Coefficient of determination R-squared

R^2 compares the model's residual variation with the variation in the observed target relative to a mean-prediction reference. The recorded XGBoost result is $R^2 = 0.7798$. Under the standard interpretation for this evaluation, the model accounts for a substantial proportion of the observed variation relative to that reference, but the value is not a direct measure of average error in knots.

R^2 should not be interpreted as "77.98 percent accurate". Accuracy in that sense is not what the statistic measures. R^2 can be high when the target has a broad range, even if absolute errors are operationally meaningful; conversely, it can be low when the target range is narrow despite modest absolute errors. This is why MAE and RMSE are reported alongside R^2.

R^2 can also be negative on a test set if the model performs worse than predicting the test-set mean, depending on the evaluation definition. Its value is affected by the target distribution and the selected evaluation subset. Therefore, a fair model comparison should report the target distribution and use aligned test examples.

The thesis uses R^2 as one component of a metric set rather than as the sole criterion. It is particularly important not to infer flight-safety readiness from R^2 alone. Operational readiness would require additional validation, threshold analysis, data-quality controls and formal acceptance by the responsible aviation stakeholders.

A.24 Circular angular error for direction

Wind-direction error must respect circular geometry. A simple difference between two angles can give a misleading result near the 0/360-degree boundary. A common angular absolute error is the smaller of the absolute difference and 360 degrees minus that difference, assuming both values are valid directions in the same convention.

For example, if the observed direction is 359 degrees and the prediction is 1 degree, the raw difference is 358 degrees, but the shortest angular separation is 2 degrees. This example illustrates why ordinary MAE on raw degrees is unsuitable for circular directions unless a circular error transformation is applied.

The recorded project direction metric is an angular MAE of 10.61 degrees. To make this result reproducible, the evaluation should state the exact circular difference formula, how calm-wind cases are handled, and whether the metric applies to a direct direction output or to a downstream direction estimate. The source code's sine/cosine features alone do not prove that a direction prediction target was trained.

Wind direction also becomes less operationally stable when wind speed is close to calm conditions. A direction value can change considerably while the wind vector magnitude remains small. Therefore, a direction metric should be interpreted with wind-speed context, and any exclusion of calm or near-calm observations must be reported explicitly.

A.25 Runway-relative wind equations

The wind component calculates runway-relative headwind and crosswind from wind speed, wind direction and runway heading. The implementation computes the angular difference between the wind direction and runway heading, then applies cosine for the longitudinal component and sine for the lateral component. The returned crosswind magnitude is absolute, while headwind retains its sign in the current helper function.

In mathematical terms, if V is wind speed and Δ is the relative angle, the implemented components are $H = V \cos(\Delta)$ and $C = |V \sin(\Delta)|$. The crosswind magnitude is non-negative because the function takes the absolute value. The signed headwind component can distinguish a headwind from a tailwind only if the wind-direction convention and sign interpretation are correctly defined.

A worked example can demonstrate the calculation without pretending it is an observed project result. If V = 10 knots and the relative angle is 60 degrees, $H = 10 \cos(60°) = 5$ knots and $C = |10 \sin(60°)| \approx 8.66$ knots. This is a mathematical illustration only. It is not a claim that the model produced these values in a particular test case.

The implementation currently compares Runway 04 at 40 degrees and Runway 22 at 220 degrees, then selects the runway with the larger headwind according to its rule. The thesis should describe this as a deterministic software rule, not as a complete runway assignment system. Actual runway selection can depend on factors outside the component, including airport procedures and operational restrictions.

A.26 Runway selection logic and boundary cases

The runway-selection helper compares the calculated headwind values for the two configured runway headings and chooses the option with the larger headwind. This rule is understandable and repeatable, which makes it suitable for a prototype decision-support display. It does not establish that the selected runway is operationally available or that the runway is the only acceptable option.

Boundary cases should be considered in testing. When wind is aligned with a runway heading, the crosswind magnitude should approach zero and the longitudinal component should approach the wind speed under the code's angle convention. When the wind is perpendicular to the runway, the longitudinal component should approach zero and the crosswind magnitude should approach the wind speed. When the relative angle is 180 degrees, the longitudinal component becomes negative, representing the opposite longitudinal direction under the implemented convention.

If the two candidate runways have equal calculated headwind, the current code's tie behaviour depends on the exact comparison and ordering in the implementation. That behaviour should be inspected and documented rather than assumed. Similarly, missing or non-numeric wind inputs should be rejected or handled safely instead of being allowed to produce misleading values.

The thesis should avoid presenting runway choice as a fully validated aviation procedure. It is an algorithmic interpretation of the model's wind forecast. A production workflow would need confirmation of runway identifiers and headings, direction conventions, availability data, operational rules and stakeholder approval.

A.27 Crosswind alert logic

The wind router contains alert logic based on crosswind magnitude. The documented code returns a high-crosswind status when crosswind exceeds 15 knots and a severe-crosswind status when it exceeds 25 knots. Since the second threshold is larger, the ordering of the conditions matters: the severe condition must be checked before a more general high condition if the intention is to preserve the severe label at values above 25 knots.

These thresholds are implementation constants in the project. The thesis should not label them as universal limits for every aircraft type, operator or airport. Aircraft limitations and operational decisions vary, and the current system does not establish certification or approval for operational dispatch. The alert is best described as a prototype informational indicator that highlights forecast crosswind magnitude.

A threshold test should include values below, exactly at and above each boundary. Because the code uses greater-than comparisons, a value exactly equal to 15 does not satisfy “greater than 15”, and a value exactly equal to 25 does not satisfy “greater than 25”. These boundary behaviours should be stated clearly in test documentation. Test cases should also cover negative or missing inputs and verify that the input validation layer prevents invalid values from producing an alert.

The recorded XGBoost test set had no observations above the configured 15- or 25-knot crosswind thresholds. Therefore, the absence of high/severe cases in that test set does not validate the alert logic under extreme crosswind. The software conditions can be unit-tested with synthetic inputs, but synthetic unit tests are not evidence of real-world extreme-weather model accuracy.

A.28 Forecast target time and horizon semantics

A forecast is meaningful only when its target time is unambiguous. The project includes one-hour and three-hour wind forecast routes, including XGBoost, TFT and hybrid labels. The target time is stored separately from the predicted weather values in the data model. This supports the distinction between the time at which a forecast was generated and the future time for which it is valid.

A forecast verification record should match a prediction to an observation representing the same target time and compatible weather variable. If the observation is several minutes away from the target time, the verification process needs a defined tolerance. If the same prediction is matched to multiple observations, the resulting error summary may be biased by duplicated records. If one forecast type is verified against a different set of target times than another, a direct metric comparison becomes less reliable.

The project verification export contains labels such as 1H, 3H, TFT_1H, TFT_3H and HYBRID_3H. It also contains pending-live-data rows and duplicate-looking target entries. Those rows should be reconciled before the export is used as the official result table. The target-time matching rule, tolerance and duplicate-removal policy should be written down and applied consistently.

The one-hour and three-hour horizons should not be treated as interchangeable. Prediction difficulty usually changes with horizon, but the magnitude of that change must be measured on comparable evaluation windows. The thesis therefore reports model configuration and project-level paper claims separately from preliminary verification-export summaries.

A.29 Verification record schema

The project database defines forecast and verified-forecast structures. A forecast record includes a target time and predicted weather fields, while the verified record includes observed weather attributes such as dry temperature, wind speed, wind direction, relative humidity, clouds, visibility and QNH, together with headwind, crosswind and remarks. This structure provides a basis for matching predictions with subsequent observations and storing operationally interpretable values.

A reliable verification record should preserve enough information to reconstruct the comparison: model identifier, prediction generation time, target time, predicted value, observed value, units, error, observation source and verification status. If the current schema does not store every item, some can be derived or added in a future revision. The thesis should distinguish fields present in the implementation from recommended audit fields.

Pending rows are not numerical observations and should not be included in MAE or RMSE calculations. Likewise, duplicate rows should be investigated before removal, because duplicates may indicate repeated ingestion or legitimate multiple forecasts for the same target. A duplicate key should be based on the business meaning of the record, such as forecast type and target time, rather than on a casual visual similarity alone.

The available verification text demonstrates that the project has a manual or exported comparison workflow, but it is not sufficient by itself to establish a fully cleaned final benchmark. The final thesis should either attach a reproducible cleaned export or explicitly identify the export as preliminary evidence.

A.30 Verification export: interpretation limits

The supplied verification export contains observed and predicted wind speeds for several forecast labels. Preliminary non-pending counts recorded from the export were 38 rows for 1H, 38 for 3H, 29 for TFT_1H, 31 for TFT_3H and 15 for HYBRID_3H. These counts are descriptive of the parsed export at the time of inspection; they should not be treated as final evaluation sample sizes until duplicates and pending entries are reconciled.

The preliminary mean absolute errors calculated from the listed rows were approximately 0.9255 knots for 1H, 1.9205 knots for 3H, 1.24 knots for TFT_1H, 1.6523 knots for TFT_3H and 1.0013 knots for HYBRID_3H. These are exploratory summaries only. They differ in sample count and may include duplicated target times, so they are not appropriate as official model rankings. In particular, the preliminary hybrid average is not identical to the paper's "below 1.00 knot" claim.

The export includes rows marked PENDING LIVE DATA. Such rows cannot contribute to an observed-error calculation until the matching observation becomes available. A robust verification script should parse numeric observations, reject pending values, detect duplicates under a documented key, compute absolute errors and output both metric values and sample counts.

The main text should prioritize the recorded XGBoost held-out metrics and clearly label TFT/hybrid figures as paper-reported where appropriate. This prevents an exploratory operational log from being mistaken for a controlled test-set experiment.

A.31 Backend route separation

The backend wind router exposes separate paths for forecast horizons and model types. The documented routes include one-hour prediction, three-hour prediction, TFT three-hour prediction and hybrid three-hour prediction. Keeping these paths separate allows the frontend or pilot workflow to request a specific forecast type and makes it easier to compare outputs during development.

A route handler generally needs to validate the request, load or call the appropriate model service, return a forecast and attach the target time. The actual endpoint contract should be documented from the submitted source code, including HTTP method, URL path, required query parameters, response fields and error responses. The thesis should not invent a URL or response example that does not match the implementation version.

Separation of routes also introduces consistency requirements. The different model endpoints should use the same units, timestamp convention and naming for comparable fields. If a frontend displays multiple model outputs, it should label each model and horizon clearly. If one route fails or has no prediction for a target time, the system should display that state instead of silently substituting a value from another model.

The existence of routes demonstrates backend integration, but does not by itself prove load capacity, availability or operational reliability. Those properties require performance and reliability testing under defined conditions. The current thesis therefore describes the implemented architecture and identifies production-scale testing as future work unless such test evidence is supplied.

A.32 Pilot portal workflow and flight-plan data

The pilot router accepts flight-plan information including departure, destination, departure time, area, flight levels, map selections, a pilot reference, flight number and preview-related fields. The code also includes functions for retrieving live maps, fetching TAFs, generating a briefing PDF and storing flight-plan records. These functions show how the broader platform connects weather information with a pilot-facing workflow.

This thesis treats the pilot portal as the presentation and integration layer for the individual wind component, rather than claiming that the author individually developed every pilot-portal function. The individual contribution is the availability of wind forecasts and runway-relative interpretation through the backend, together with the connection points used by the broader pilot workflow.

A user workflow can be described as: provide flight-plan details; request relevant weather information; retrieve the selected forecast/map products; generate or display a briefing; and retain the flight-plan record where configured. The actual sequence and visible user experience should be confirmed from the deployed frontend and screenshots. The presence of backend methods does not prove that every UI step is exposed or functioning in the current deployment.

For final evidence, include screenshots from the real portal and annotate only elements that are visible in the submitted build. Do not use mockups as evidence of implemented features. If a feature is present in backend code but not exposed in the frontend, label it as backend capability or incomplete integration rather than a completed user-facing feature.

A.33 Scheduling and data refresh

The project scheduler configures periodic tasks for live METAR acquisition, model retraining and maintenance. The recorded settings include live METAR fetches every 32 minutes, monthly retraining on day one at 00:00, a daily 48-hour backup at 01:00, an initial fetch at startup, daily log cleanup after 30 days and temporary map cleanup after one day. These timings are code-level configuration evidence and should be checked against the final deployed configuration before being described as active production behaviour.

Scheduling reduces the need for manual execution, but it creates operational dependencies. A scheduled fetch can fail because of network availability, an upstream service issue, authentication or data-format changes. A retraining job can fail if the database is incomplete or the new model performs worse than the existing one. For that reason, a scheduler should log task start, completion, outcome and errors, and should not replace a known-good model with an unvalidated candidate.

The project retrainer evaluates a new model against the old model and returns test MAE. This supports a model-maintenance process, but the thesis should only claim automated promotion or rollback if the actual code performs those actions. Evaluation and replacement are separate steps.

A useful operational checklist includes checking the latest observation timestamp, ingestion status, backup completion, model version, last training result and forecast freshness. These are recommended monitoring checks; they should be labelled as recommendations unless the user interface or log evidence confirms that they are already implemented.

A.34 Model retraining and model governance

Retraining is intended to refresh a model as additional observations become available. The project includes a monthly retraining schedule and a wind retrainer that evaluates a new candidate against the existing model. This is a practical starting point for model maintenance because it recognizes that a model should not be assumed to remain equally effective indefinitely.

A responsible retraining process should preserve the current production model until the candidate passes validation. It should record the training data cutoff, preprocessing version, hyperparameters, evaluation metrics and artifact identifier. If the candidate performs worse than the existing model, the system should retain the existing model and record the rejection reason. The thesis should describe only the behaviours that are implemented in the repository and classify additional safeguards as recommendations.

Weather data can change in distribution over time because of observation practices, equipment, data availability or atmospheric conditions. A drift signal is not automatically proof that retraining is needed; it should be interpreted with data-quality checks and model-performance evidence. Retraining on corrupted or inconsistent inputs may worsen performance rather than improve it.

The project notes mention background retraining and model maintenance. To make the claim auditable, final submission evidence should include the retraining function, scheduler configuration and representative logs or result files. Without those logs, the thesis can explain the implemented design but should not claim that the process has run successfully for every scheduled cycle.

A.35 API error handling and input validation

A.36 Pilot-facing communication and safe interpretation

A.37 Software test strategy

A.38 Test cases: runway calculations

A.39 Test cases: data and model pipeline

A.40 Test cases: API and pilot workflow

A.41 Non-functional requirements and evidence

A.42 Traceability from objectives to evidence

A.43 Limitations of the available evidence

A.44 Threats to validity

A.45 Reproducibility checklist

A.46 Data integrity and database consistency

A.47 Missing data and forecast availability

A.48 Model comparison protocol

A.49 Operational monitoring indicators

A.50 Risk register for the wind component

A.51 Privacy, access control and responsible use

A.52 Sustainability and maintainability of the implementation

A.53 Commercial and deployment boundary

A.54 Future evaluation plan

A.55 Reproducible pseudocode for the wind workflow

A.56 Research reproducibility and version control

A.57 Interpretation of the recorded XGBoost results

A.58 Interpreting no observed extreme-threshold cases

A.59 Research questions and evidence mapping

A.60 Glossary of component-specific terms

Appendix B: Academic integrity and AI use disclosure